

THE ANALYST HANDBOOK SERIES

Data Fundamentals, SQL & Python

Volume 1 — Semester 1 dari Kurikulum 7-Layer Analytics

Handbook teknis premium untuk membangun fondasi analytics kelas produksi: dari data literacy, SQL tingkat produksi, arsitektur data, data preparation, hingga analytical dataset yang dapat dipercaya.

■ **LAYER 1 — DATA** (volume ini)

☐ LAYER 2 — STATISTICS

☐ LAYER 3 — BUSINESS

☐ LAYER 4 — EXPERIMENTATION

☐ LAYER 5 — DECISION SCIENCE

☐ LAYER 6 — AI

☐ LAYER 7 — DOMAIN EXPERTISE

Tentang Buku Ini

Cara membaca, standar yang dipakai, dan peta perjalanan tujuh semester

Untuk siapa buku ini ditulis

Buku ini ditulis untuk calon data analyst yang ingin mencapai standar kerja korporasi besar. Bukan sekadar bisa menulis query. Targetnya adalah analyst yang mampu mengubah data mentah menjadi evidence, evidence menjadi keputusan, dan keputusan menjadi business impact.

Standar penulisannya mengikuti pola pendidikan teknik intensif: padat, terstruktur, berbasis praktik industri, dan menuntut penguasaan penuh sebelum naik level. Setiap konsep diajarkan sampai tuntas — termasuk kesalahan umum, cara senior menghindarinya, dan versi produksinya.

Struktur setiap bab

Setiap bab mengikuti urutan yang sama dan tidak pernah berubah:

Urutan	Bagian	Fungsi
1	WHY	Mengapa konsep ini ada dan masalah bisnis apa yang diselesaikannya
2	WHAT	Definisi presisi dan terminologi standar
3	WHEN	Kapan digunakan, kapan tidak
4	HOW	Langkah teknis, kode, dan contoh bertingkat: simple → business → production
5	BEST PRACTICE	Standar industri dan kebiasaan senior
6	COMMON MISTAKE	Kesalahan umum, gejalanya, dan cara memperbaikinya
7	CHECKLIST	Validasi sebelum hasil dianggap selesai
8	ENDING CARD	Summary, exercises + answer key, interview questions, portfolio idea

Callout boxes

Sepanjang buku, informasi penting ditandai dengan kotak berwarna yang konsisten:

◆ BUSINESS INSIGHT

Menghubungkan konsep teknis ke konsekuensi bisnis.

■ TECHNICAL NOTE

Detail teknis yang perlu dipahami tetapi tidak mengganggu alur utama.

✓ BEST PRACTICE

Standar kerja yang dipakai tim analytics matang.

✗ COMMON MISTAKE

Kesalahan yang berulang kali muncul di dunia nyata — beserta perbaikannya.

⚠ WARNING

Hal yang bisa menyebabkan angka salah atau keputusan keliru.

▲ PERFORMANCE TIP

Cara membuat query dan proses berjalan cepat di data besar.

● INTERVIEW TIP

Bagaimana topik ini diuji dalam interview data analyst.

★ PORTFOLIO TIP

Cara mengubah materi bab menjadi bukti kemampuan yang bisa ditunjukkan.

► EXECUTIVE PERSPECTIVE

Bagaimana eksekutif membaca dan menggunakan hasil kerja analyst.

Peta 7 semester

Sem	Layer	Pertanyaan utama	Output utama
1	Data ← volume ini	Apa yang terjadi?	Analytical dataset yang dapat dipercaya
2	Statistics + Visualization	Seberapa kuat / pasti?	Statistical analysis & BI dashboard
3	Business	Mengapa ini penting?	KPI, unit economics, root-cause analysis
4	Experimentation	Apakah X menyebabkan Y?	A/B test & causal analysis
5	Decision Science	Apa yang sebaiknya dilakukan?	Decision model & resource allocation
6	AI	Bagaimana memperbesar kemampuan?	AI-assisted & predictive analytics
7	Domain + Capstone	Bagaimana bisnis bekerja?	Business Decision System

Dataset yang dipakai sepanjang volume ini

Semua contoh menggunakan satu perusahaan fiktif yang realistis: **TokoNusa**, retail omnichannel Indonesia dengan 85 toko fisik dan e-commerce, $\pm 2,1$ juta pelanggan terdaftar, dan ± 40.000 transaksi per hari. Skema datanya diperkenalkan di Bab 2 dan dipakai konsisten sampai capstone. Menggunakan satu perusahaan yang sama membuat setiap konsep baru langsung terasa kontekstual — seperti bekerja di perusahaan sungguhan sejak hari pertama.

Daftar isi Volume 1

- Bab 1** Cara Berpikir Seorang Analyst — analytics value chain, business-first mindset
- Bab 2** Data Literacy — tipe data, grain, keys, integrity, lineage
- Bab 3** SQL Fundamental — SELECT sampai JOIN, agregasi, CASE
- Bab 4** SQL Tingkat Produksi — subquery, CTE, window function, date logic

- Bab 5** Arsitektur Data & Pipeline — OLTP/OLAP, star schema, ETL/ELT, pipeline mindset
- Bab 6** Data Preparation — cleaning framework, missing values, outlier, feature creation
- Bab 7** Excel & Python untuk Analyst — kapan pakai apa, pandas workflow
- Bab 8** Capstone Semester 1 — analytical dataset end-to-end + dokumentasi + komunikasi

Bab 1 — Cara Berpikir Seorang Analyst

Sebelum menyentuh SQL: memahami untuk apa semua ini ada

Learning objectives	Memahami analytics value chain; membedakan analyst pelapor angka vs analyst penggerak keputusan; menguasai alur business question → data → insight → rekomendasi
Prerequisites	Tidak ada
Estimasi waktu	Baca 45 menit · Praktik 60 menit
Difficulty	● Beginner
Skills covered	Problem framing dasar, analytics workflow, stakeholder awareness
Business domain	Retail (TokoNusa)

Mengapa bab ini ada sebelum bab teknis

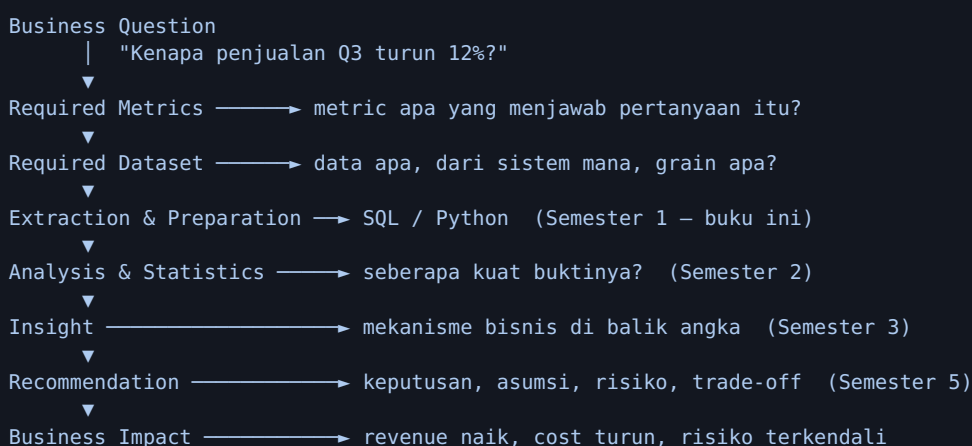
Sebagian besar analyst gagal bukan karena tidak bisa menulis query. Mereka gagal karena menjawab pertanyaan yang salah dengan presisi tinggi.

Query yang sempurna untuk pertanyaan yang keliru menghasilkan keputusan yang keliru. Karena itu cara berpikir diajarkan lebih dulu daripada tool.

Prinsip inti kurikulum ini: **tool bukan tujuan**. SQL, Python, dan Power BI adalah alat untuk menghasilkan analisis yang dapat dipercaya dan keputusan yang berdampak.

Apa itu analytics value chain

Seluruh pekerjaan analytics bergerak melalui satu rantai nilai. Setiap semester dalam kurikulum ini memperkuat satu mata rantai.



Perhatikan arahnya. Rantai selalu dimulai dari pertanyaan bisnis, bukan dari data. Analyst yang memulai dari data akan menghasilkan laporan. Analyst yang memulai dari pertanyaan akan menghasilkan keputusan.

◆ BUSINESS INSIGHT

Perusahaan tidak membayar analyst untuk query. Perusahaan membayar untuk pengurangan ketidakpastian dalam pengambilan keputusan. Query hanyalah cara mencapainya.

Dua tipe analyst — dan mengapa hanya satu yang dipromosikan

	Report Producer	Decision Driver
Menerima permintaan	"Buatkan laporan penjualan" → langsung mengerjakan	Bertanya dulu: keputusan apa yang akan diambil dari laporan ini?
Output	Tabel dan grafik	Insight + rekomendasi + risiko
Ketika angka aneh	Melaporkan apa adanya	Menelusuri sebab sebelum melaporkan
Ukuran sukses	Laporan selesai tepat waktu	Keputusan yang lebih baik terjadi
Karier	Tergantikan oleh dashboard otomatis dan AI	Semakin dibutuhkan di level manajemen

Seluruh buku ini melatih tipe kedua. Bahkan di bab paling teknis, setiap contoh selalu dimulai dari pertanyaan bisnis.

Kapan analyst dibutuhkan — siapa yang meminta, dan mengapa

Dalam korporasi nyata, permintaan analisis datang dari pola yang bisa diprediksi:

Pemberi tugas	Permintaan khas	Frekuensi	Deadline khas
CFO / Finance	"Margin turun, di mana bocornya?"	Bulanan + ad-hoc	2-5 hari kerja
CMO / Marketing	"Kampanye mana yang benar-benar menghasilkan?"	Per kampanye	1-3 hari kerja
COO / Operations	"Kenapa stok kosong padahal gudang penuh?"	Mingguan	1-2 hari kerja
CEO / BOD	"Kondisi bisnis sebenarnya seperti apa?"	Kuartalan	Sebelum board meeting
Product / Sales lead	"Fitur/promo ini berhasil tidak?"	Sprint / per event	Jam-jam, bukan hari

Perhatikan kolom deadline. Kecepatan adalah bagian dari kualitas. Analisis sempurna yang datang setelah keputusan diambil bernilai nol. Ini sebabnya fondasi SQL dan data preparation di semester ini dilatih sampai otomatis — kecepatan teknis membeli waktu untuk berpikir.

Bagaimana menerima permintaan analisis dengan benar

Sebelum menulis satu baris query pun, jawab empat pertanyaan ini. Empat pertanyaan ini akan menjadi refleksi seumur karier.

1. **Keputusan apa yang bergantung pada analisis ini?** Jika tidak ada keputusan, tanyakan kembali urgensinya.
2. **Siapa yang akan membaca hasilnya?** CFO membutuhkan angka dan risiko. Store manager membutuhkan tindakan konkret.

3. **Definisi metric-nya apa?** "Penjualan" bisa berarti gross revenue, net revenue, atau jumlah transaksi. Salah definisi = analisis ulang dari nol.
4. **Kapan hasilnya harus siap, dan seberapa presisi harus?** Estimasi kasar hari ini sering lebih berharga daripada angka presisi minggu depan.

✗ COMMON MISTAKE — Langsung menulis query

Gejala: analisis selesai, lalu stakeholder berkata "bukan itu maksud saya."

Mengapa terjadi: permintaan bisnis hampir selalu ambigu; yang meminta sering belum tahu persis apa yang mereka butuhkan.

Perbaikan: tulis ulang permintaan dalam satu kalimat + definisi metric, kirim untuk konfirmasi sebelum mulai. Kebiasaan 5 menit ini menyelamatkan hari-hari kerja.

Cara senior menghindarinya: senior selalu membalas permintaan dengan pertanyaan "keputusan apa yang mau diambil?" — bukan karena sok kritis, tetapi karena sudah pernah membuang minggu-minggu untuk analisis yang tidak dipakai.

Contoh lengkap: dari permintaan kabur ke rencana analisis

Level 1 — Simple example

Permintaan: "*Berapa penjualan bulan lalu?*"

Klarifikasi analyst: penjualan = net revenue setelah retur? Bulan kalender atau 30 hari terakhir? Semua channel? Hasil klarifikasi menjadi spesifikasi: *net revenue Juli 2026, semua channel, exclude retur dan pembatalan.*

Level 2 — Business example

Permintaan CMO TokoNusa: "*Promo payday kemarin sukses nggak sih?*"

Analyst menerjemahkan menjadi pertanyaan terukur:

- Apakah revenue selama periode promo lebih tinggi dari baseline periode setara tanpa promo?
- Apakah kenaikan berasal dari pelanggan baru atau pelanggan lama yang mempercepat pembelian (pull-forward)?
- Berapa margin setelah biaya diskon — untung atau sekadar ramai?

Tiga pertanyaan ini menentukan dataset yang harus dibangun: transaksi harian $\pm$ 8 minggu, flag pelanggan baru/lama, dan biaya promo. Tanpa penerjemahan ini, analyst hanya akan melaporkan "revenue naik 20%" — angka yang terlihat bagus tetapi tidak menjawab apakah promo layak diulang.

Level 3 — Production example

Di perusahaan matang, penerjemahan itu didokumentasikan sebagai **analysis brief** singkat sebelum pengerjaan:

```
ANALYSIS BRIEF — Evaluasi Promo Payday Juli
Requestor      : CMO                               Deadline: Kamis 16:00
Decision       : Apakah promo payday diulang bulan depan dengan budget sama?
Metrics        : incremental net revenue, margin after discount,
                % new customer, repeat-rate D+30
Dataset        : sales_transactions (grain: item per order),
                customers, promo_costs — periode 1 Jun–31 Jul
Asumsi         : baseline = rata-rata 4 payday week sebelumnya
Out of scope   : efek jangka panjang ke brand (butuh eksperimen, Semester 4)
```


Perhatikan baris terakhir. Menyatakan apa yang *tidak* dijawab sama pentingnya dengan apa yang dijawab. Ini melindungi analyst dari over-claim dan melatih stakeholder memahami batas evidence.

► EXECUTIVE PERSPECTIVE

Eksekutif membaca hasil analisis dalam 90 detik pertama: kesimpulan, angka kunci, rekomendasi. Detail metodologi hanya dibaca jika kesimpulannya mengejutkan. Struktur komunikasi ini — kesimpulan dulu, detail belakangan — disebut *pyramid principle* dan dilatih penuh di Bab 8.

Standar bukti: hierarki kepercayaan angka

Tidak semua angka layak dipercaya sama besar. Sejak semester ini, biasakan menilai setiap angka dengan hierarki berikut:

TINGKAT KEPERCAYAAN (naik ke atas)

▲ 5. Hasil eksperimen terkontrol (A/B test)	→ Semester 4
4. Analisis kausal dengan kontrol confounder	→ Semester 4
3. Metric tervalidasi dari pipeline teruji	→ Semester 1–2
2. Query ad-hoc yang direview orang lain	→ Semester 1
1. Query ad-hoc tanpa review, definisi tidak jelas	→ sumber insiden

Semester 1 bertugas menaikkan semua pekerjaanmu minimal ke level 3: angka yang keluar dari proses yang terdefinisi, tervalidasi, dan bisa direproduksi. Bab 5 dan 6 membangun kebiasaan itu.

Best practice: kebiasaan harian analyst elite

- ✓ Selalu tulis ulang permintaan menjadi pertanyaan terukur sebelum mengerjakan.
- ✓ Simpan semua query dan script — pekerjaan yang tidak bisa direproduksi dianggap tidak pernah terjadi.
- ✓ Validasi angka dengan sumber kedua sebelum dikirim (rekonsiliasi sederhana: total revenue vs laporan finance).
- ✓ Sertakan definisi metric di setiap deliverable. Satu baris footnote mencegah satu jam rapat.
- ✓ Catat asumsi. Angka tanpa asumsi tertulis adalah angka yang tidak bisa dipertanggungjawabkan.
- ✓ Kirim hasil sebelum deadline dengan estimasi kasar lebih dulu jika perlu — kecepatan membangun kepercayaan.

Checklist sebelum memulai analisis apa pun

- ✓ Keputusan yang bergantung pada analisis ini sudah jelas
- ✓ Definisi metric sudah dikonfirmasi tertulis
- ✓ Audiens dan format output sudah diketahui
- ✓ Deadline dan tingkat presisi sudah disepakati
- ✓ Scope dan out-of-scope sudah dinyatakan

Ending Card — Bab 1

What you learned

Analytics value chain dari business question sampai business impact; perbedaan report producer vs decision driver; empat pertanyaan wajib sebelum analisis; analysis brief; hierarki kepercayaan angka.

Skills mastered

Problem framing dasar · penerjemahan permintaan ambigu → spesifikasi terukur · penulisan analysis brief.

Real business value

Mengurangi rework, mempercepat time-to-decision, dan membangun reputasi sebagai analyst yang bisa dipercaya — aset karier paling awal dan paling tahan lama.

Interview readiness

● INTERVIEW QUESTIONS

1. "Stakeholder meminta 'laporan penjualan'. Apa yang kamu lakukan pertama kali?" — jawaban kuat: klarifikasi keputusan, definisi metric, audiens, deadline.
2. "Ceritakan analisis yang kamu kerjakan tapi tidak dipakai. Kenapa?" — pewawancara menguji apakah kamu memahami akar masalahnya: framing, bukan teknis.
3. "Bagaimana kamu memastikan angkamu benar sebelum dikirim ke management?" — jawaban kuat menyebut rekonsiliasi dengan sumber kedua dan review.

Exercises

Exercise 1.1 — Menerjemahkan permintaan (Beginner · 15 menit)

Objective: mengubah permintaan ambigu menjadi spesifikasi terukur

Terjemahkan tiga permintaan berikut menjadi pertanyaan terukur + definisi metric + dataset yang dibutuhkan: (a) "Pelanggan kita loyal nggak sih?" (b) "Toko cabang Bandung kok sepi ya?" (c) "Produk baru kita laku nggak?"

Answer key 1.1

(a) Loyal → repeat purchase rate: % pelanggan dengan ≥ 2 transaksi dalam 90 hari; dataset: transaksi per customer_id 6 bulan terakhir. (b) Sepi → bandingkan traffic, transaksi, dan conversion vs baseline cabang serupa dan vs periode sebelumnya; dataset: transaksi + footfall per toko per hari. (c) Laku → unit terjual dan sell-through vs target dan vs produk pembanding di kategori sama sejak launch; dataset: sales per SKU per minggu + stok awal. Kesalahan umum: menjawab dengan satu angka absolut tanpa pembanding — angka tanpa baseline tidak bisa diinterpretasikan.

Exercise 1.2 — Menulis analysis brief (Intermediate · 30 menit)

Objective: menghasilkan analysis brief lengkap

COO TokoNusa bertanya: "Gudang Surabaya kayaknya lambat, bener nggak?" Tulis analysis brief lengkap: decision, metrics, dataset, asumsi, out-of-scope, deadline yang masuk akal.

Answer key 1.2 (contoh jawaban kuat)

Decision: apakah perlu menambah shift atau merombak proses picking di gudang Surabaya. Metrics: lead time order-to-ship (median & P90), throughput per hari, backlog harian — dibandingkan gudang Jakarta sebagai baseline dan vs 8 minggu sebelumnya. Dataset: order fulfillment log per order (grain: order), 12 minggu. Asumsi: mix produk kedua gudang setara (perlu dicek). Out-of-scope: akar masalah proses internal (butuh observasi lapangan). Alternatif jawaban yang juga benar: menjadikan perbandingan mix produk bagian dari analisis, bukan asumsi.

Portfolio idea

★ PORTFOLIO TIP

Mulai repositori portofolio sekarang. Artefak pertama: tiga analysis brief dari kasus berbeda (retail, e-commerce, F&B). Recruiter jarang melihat kandidat junior yang bisa menunjukkan cara berpikir sebelum kode — ini pembeda instan.

Further reading

Barbara Minto, *The Pyramid Principle* · Google Data Analytics: "Ask" phase · Thinking with Data (Max Shron), bab "Scoping".

Next chapter

Bab 2 membangun kosakata data: tipe data, grain, key, dan integrity — bahasa yang dipakai seluruh sisa buku ini.

Bab 2 — Data Literacy

Bahasa yang dipakai seluruh dunia data — kuasai sebelum menyentuh query

Learning objectives	Menguasai tipe data dan skala pengukuran; memahami grain, dimension vs measure, primary/foreign key dan relationships; mengenali NULL, duplicate, dan ancaman data integrity; membaca data lineage
Prerequisites	Bab 1
Estimasi waktu	Baca 90 menit · Praktik 90 menit
Difficulty	● Beginner
Skills covered	Data types, NOIR scale, grain analysis, keys & relationships, integrity, lineage
Business domain	Retail (TokoNusa)

Mengapa data literacy menentukan segalanya

Setiap kesalahan analisis yang besar hampir selalu berakar pada kesalahan literacy yang kecil: salah memahami grain sehingga revenue terhitung ganda, salah memperlakukan NULL sehingga rata-rata bias, atau salah membaca relasi tabel sehingga JOIN meledakkan baris.

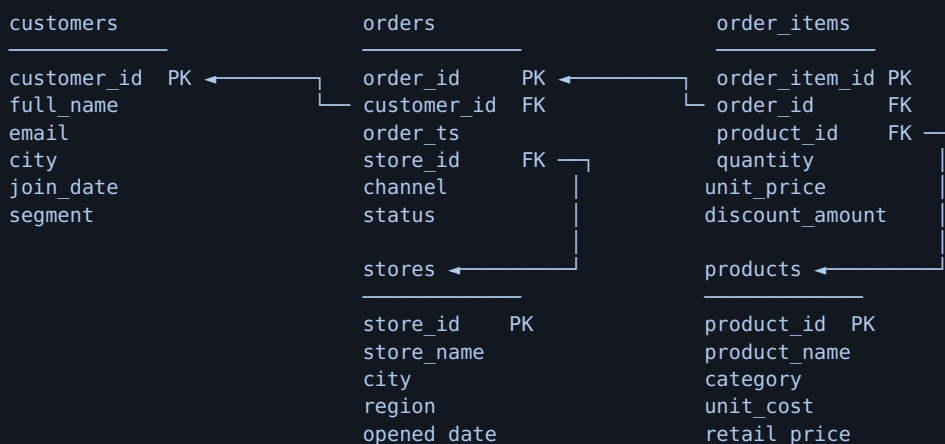
Kesalahan ini tidak menghasilkan error message. Query berjalan sukses, angkanya keluar, dan angkanya salah. Itulah yang membuatnya berbahaya.

⚠ WARNING

Kesalahan paling mahal dalam analytics bukan query yang gagal, melainkan query yang berhasil dengan hasil yang salah. Bab ini adalah sistem imun terhadap kesalahan jenis itu.

Dataset TokoNusa — skema yang dipakai sepanjang buku

Lima tabel inti yang akan kamu pakai dari sini sampai capstone:



■ TECHNICAL NOTE

PK = primary key, FK = foreign key — dijelaskan lengkap di bagian keys. Panah menunjukkan arah referensi: satu customer memiliki banyak orders; satu order memiliki banyak order_items. Pola ini disebut relasi one-to-many dan merupakan struktur paling umum di data transaksional.

Tipe data: categorical vs numerical

Mengapa pembagian ini penting

Tipe data menentukan operasi apa yang bermakna. Menjumlahkan `store_id` secara teknis bisa, secara makna absurd. Software tidak akan melarangmu — pemahamanmu yang harus melarang.

Apa definisinya

Tipe	Definisi	Contoh di TokoNusa	Operasi bermakna
Categorical	Label keanggotaan kelompok	channel, city, segment, status	count, proportion, mode, grouping
Numerical	Kuantitas yang bisa dihitung	quantity, unit_price, discount	sum, mean, median, min/max, korelasi

✗ COMMON MISTAKE — Angka yang bukan kuantitas

Gejala: laporan menampilkan "rata-rata store_id = 43,7" atau kode pos dijumlahkan.

Mengapa terjadi: kolom bertipe angka di database otomatis dianggap kuantitas. Padahal store_id, kode pos, dan nomor telepon adalah *identifier* — categorical yang kebetulan ditulis dengan digit.

Perbaikan: uji dengan pertanyaan: "apakah dua kali nilainya berarti dua kali sesuatu?" Jika tidak, perlakukan sebagai categorical.

Cara senior menghindarinya: senior membaca data dictionary dulu, dan menyimpan identifier sebagai string di analytical dataset agar tidak tergoda dihitung.

Empat skala pengukuran (NOIR)

Pembagian categorical/numerical bisa diperhalus menjadi empat skala. Urutannya membentuk akronim NOIR — dari informasi paling sedikit ke paling kaya.

Skala	Sifat	Contoh	Statistik yang valid
Nominal	Label tanpa urutan	channel (online/offline), city	count, mode, proportion
Ordinal	Berurutan, jarak tak terukur	segment (Bronze<Silver<Gold), rating 1-5	+ median, percentile
Interval	Jarak bermakna, nol arbitrer	suhu °C, tanggal	+ mean, selisih
Ratio	Nol mutlak, rasio bermakna	revenue, quantity, umur	+ rasio ("2× lebih besar")

Konsekuensi praktis yang paling sering muncul: rating kepuasan 1-5 adalah ordinal. Rata-rata rating 4,2 secara teknis menyalahi skala (jarak antar level tidak terukur), tetapi industri tetap memakainya sebagai konvensi. Yang penting: laporkan juga distribusinya, karena rata-rata 4,2 bisa berarti "mayoritas 4-5" atau "banyak 5 dan banyak 1" — dua situasi bisnis yang sangat berbeda.

● INTERVIEW TIP

"Apa bedanya data ordinal dan interval? Beri contoh." adalah pertanyaan screening klasik. Jawaban yang membedakan kandidat: menyebut konsekuensinya terhadap pilihan statistik, bukan hanya definisi.

Grain: konsep terpenting di seluruh bab ini

Mengapa

Grain adalah jawaban atas pertanyaan: **satu baris di tabel ini mewakili apa?** Semua agregasi, JOIN, dan metric bergantung pada jawaban itu. Salah memahami grain adalah sumber kesalahan #1 analyst pemula — dan penyebab angka double-counting yang lolos sampai board meeting.

Apa

Tabel	Grain (1 baris = ?)	Konsekuensi
customers	1 pelanggan	COUNT(*) = jumlah pelanggan
orders	1 order (bisa berisi banyak item)	COUNT(*) = jumlah transaksi
order_items	1 item dalam 1 order	revenue dihitung di sini, bukan di orders

Contoh kesalahan grain yang realistis

Order O-1001 berisi 3 item. Analyst men-JOIN `orders` ke `order_items` lalu menghitung `COUNT(order_id)` untuk "jumlah transaksi". Hasilnya 3, padahal transaksinya 1. Di skala TokoNusa (40.000 order/hari, rata-rata 2,6 item/order), "jumlah transaksi" akan overstated $\pm 160\%$ — dan tidak ada error message apa pun.

✓ BEST PRACTICE

Ritual wajib sebelum memakai tabel baru: (1) tanyakan grain-nya, (2) buktikan dengan query — hitung total baris vs distinct key, (3) tulis grain di dokumentasi. Tiga menit ini mencegah kategori kesalahan terbesar di profesi ini. Query pembuktiannya diajarkan di Bab 3.

Dimension vs measure

Setelah grain, klasifikasikan setiap kolom ke dalam dua peran:

	Dimension	Measure
Fungsi	Sudut pandang untuk memecah data ("by apa?")	Angka yang diagregasi ("berapa?")
Contoh	city, channel, category, bulan	revenue, quantity, discount
Dalam kalimat	"revenue per region per bulan "	" revenue per region per bulan"

Setiap permintaan analisis bisa diurai menjadi *measure by dimension*. "Bagaimana penjualan tiap cabang bulan ini?" = SUM(revenue) by store by month. Kebiasaan mengurai seperti ini membuat penulisan SQL di Bab 3 menjadi mekanis.

Primary key, foreign key, dan relationships

Mengapa

Key adalah kontrak yang membuat data bisa dihubungkan tanpa ambigu. Seluruh keandalan JOIN — dan karena itu seluruh keandalan analytical dataset — berdiri di atas key.

Apa

- **Primary key (PK):** kolom yang nilainya unik dan tidak NULL untuk setiap baris — identitas baris. Contoh: `customer_id` di `customers`.
- **Foreign key (FK):** kolom yang menunjuk PK tabel lain. `orders.customer_id` menunjuk `customers.customer_id` — artinya setiap order dimiliki pelanggan yang sah.
- **Composite key:** kombinasi kolom yang bersama-sama unik, misalnya (`order_id`, `product_id`) jika satu produk hanya boleh muncul sekali per order.

Tiga jenis relationship

Jenis	Contoh	Risiko saat JOIN
One-to-one	<code>customer ↔ loyalty_account</code>	Aman
One-to-many	<code>customer → orders</code>	Baris sisi "many" menggandakan sisi "one" — sadari grain hasil JOIN
Many-to-many	<code>products ↔ promotions</code> (1 produk banyak promo, 1 promo banyak produk)	Fan-out eksplosif: baris hasil = perkalian. Wajib lewat bridge table

⚠ WARNING — JOIN many-to-many

JOIN langsung dua tabel yang berelasi many-to-many menghasilkan perkalian baris (cartesian fan-out). Revenue bisa membengkak beberapa kali lipat tanpa error. Deteksinya sederhana: bandingkan total measure sebelum dan sesudah JOIN — harus sama. Teknik pengecekan ini dipraktikkan di Bab 4.

NULL: ketiadaan yang punya banyak arti

Mengapa NULL berbahaya

NULL bukan nol dan bukan string kosong. NULL berarti "tidak ada nilai" — dan itu bisa bermakna sangat berbeda tergantung sebabnya:

Makna NULL	Contoh	Perlakuan yang tepat
Belum terjadi	<code>delivered_date</code> pada order yang masih dikirim	Biarkan NULL; filter by status
Tidak berlaku	<code>store_id</code> pada order online	Biarkan; jangan diisi paksa
Tidak dicatat	city kosong karena form lama tidak mewajibkan	Selidiki polanya (Bab 6)
Bug pipeline	<code>unit_price</code> tiba-tiba NULL sejak tanggal tertentu	Eskalasi ke data engineering

Perilaku NULL yang wajib dihafal

- Perbandingan apa pun dengan NULL menghasilkan UNKNOWN, bukan true/false. `WHERE city = 'Jakarta'` dan `WHERE city != 'Jakarta'` sama-sama membuang baris ber-city NULL.

- Fungsi agregasi (SUM, AVG, COUNT(kolom)) *mengabaikan* NULL. AVG dari {10, NULL, 20} = 15, bukan 10.
- NULL dalam aritmetika menular: `price - NULL = NULL`.

✗ COMMON MISTAKE — "Bukan Jakarta" yang menghilangkan baris

Gejala: jumlah "pelanggan luar Jakarta" + "pelanggan Jakarta" < total pelanggan.

Mengapa terjadi: baris dengan city NULL tidak masuk ke kedua filter.

Perbaikan: putuskan eksplisit ke mana NULL masuk: `WHERE city != 'Jakarta' OR city IS NULL`, atau laporkan kategori "Unknown" terpisah.

Cara senior menghindarinya: setiap memfilter kolom nullable, senior otomatis bertanya "NULL mau dianggap apa?" dan menuliskan keputusannya.

Duplicate: satu entitas, banyak baris

Duplicate hadir dalam dua bentuk yang penanganannya berbeda:

- **Exact duplicate:** seluruh baris identik — biasanya akibat double-load pipeline. Aman dihapus.
- **Logical duplicate:** entitas sama, baris berbeda — "Budi Santoso" dan "budi santoso " dengan email sama. Ini masalah yang jauh lebih sulit dan lebih umum.

Dampak bisnisnya langsung: jumlah pelanggan overstated, CAC understated (biaya dibagi ke pelanggan yang sama dihitung dua kali), dan email campaign mengirim dua kali ke orang yang sama. Teknik deteksi dan deduplication dipraktikkan penuh di Bab 6.

Data integrity: enam dimensi kualitas

Industri menilai kualitas data melalui enam dimensi standar. Hafalkan — ini kerangka audit yang dipakai berulang sampai capstone:

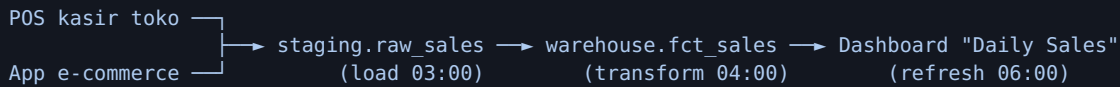
Dimensi	Pertanyaan	Contoh pelanggaran di TokoNusa
Completeness	Adakah yang kosong?	12% order tanpa customer_id
Uniqueness	Adakah duplikat?	Pelanggan sama terdaftar 2x
Validity	Sesuai aturan format/domain?	quantity = -3; email tanpa '@'
Consistency	Selaras antar sistem/kolom?	Revenue di dashboard ≠ laporan finance
Accuracy	Sesuai realitas?	Alamat toko sudah pindah, data lama
Timeliness	Cukup baru untuk keputusan?	Data stok telat 2 hari untuk keputusan restock harian

◆ BUSINESS INSIGHT

Konsistensi adalah dimensi yang paling merusak kepercayaan. Sekali eksekutif melihat dua angka revenue yang berbeda untuk periode yang sama, semua laporan berikutnya akan diragukan — dan tim analytics menghabiskan waktunya untuk rekonsiliasi, bukan analisis. Single source of truth (Bab 5) adalah jawabannya.

Data lineage: dari mana angka ini berasal

Lineage adalah jejak perjalanan data: sistem sumber → pipeline → tabel warehouse → dashboard. Analyst yang bisa menjawab "angka ini datang dari mana dan lewat transformasi apa?" dapat men-debug angka aneh dalam hitungan menit, bukan hari.



Contoh penggunaan nyata: dashboard menunjukkan penjualan kemarin anjlok 40%. Analyst yang paham lineage langsung memeriksa: apakah load 03:00 sukses? Apakah salah satu sumber (POS?) gagal kirim? Sembilan dari sepuluh "anjlok mendadak" adalah kegagalan pipeline, bukan kejadian bisnis. Melaporkan "penjualan anjlok" tanpa cek lineage adalah cara tercepat kehilangan kredibilitas.

Checklist data literacy

- ✓ Grain setiap tabel diketahui dan dibuktikan, bukan diasumsikan
- ✓ Setiap kolom terklasifikasi: dimension atau measure; categorical atau numerical
- ✓ Identifier numerik tidak diperlakukan sebagai kuantitas
- ✓ PK/FK dan arah relationship dipahami sebelum JOIN
- ✓ Keputusan perlakuan NULL eksplisit di setiap filter dan agregasi
- ✓ Duplicate dicek pada level entitas, bukan hanya exact match
- ✓ Enam dimensi integrity diperiksa sebelum data dipakai analisis
- ✓ Lineage angka penting bisa dijelaskan minimal satu tingkat ke hulu

Ending Card — Bab 2

What you learned

Tipe data dan skala NOIR; grain dan konsekuensinya; dimension vs measure; PK/FK dan tiga jenis relationship; semantik NULL; dua jenis duplicate; enam dimensi integrity; data lineage.

Skills mastered

Membaca skema database · mengurai permintaan menjadi measure-by-dimension · audit kualitas data secara konseptual.

Real business value

Mencegah kategori kesalahan paling mahal (angka salah tanpa error) dan mempercepat debugging angka aneh dari hari menjadi menit.

Interview readiness

● INTERVIEW QUESTIONS

1. "Apa itu grain? Kenapa penting?" — jawab dengan contoh double-counting.
2. "Apa hasil dari WHERE city != 'Jakarta' jika ada NULL?" — baris NULL ikut terbuang; jelaskan three-valued logic.
3. "Bagaimana kamu memverifikasi kualitas tabel yang baru pertama kamu lihat?" — jawab dengan kerangka enam dimensi + cek grain.

Exercises

Exercise 2.1 — Klasifikasi kolom (Beginner · 10 menit)

Objective: menguji refleksi tipe data

Klasifikasikan sebagai nominal/ordinal/interval/ratio dan dimension/measure: (a) order_ts, (b) segment Bronze/Silver/Gold, (c) discount_amount, (d) store_id, (e) rating 1–5.

Answer key 2.1

(a) interval, dimension (biasanya diturunkan jadi bulan/minggu). (b) ordinal, dimension. (c) ratio, measure. (d) nominal (identifier), dimension — meski berbentuk angka. (e) ordinal, bisa keduanya: dimension saat memecah ("berapa order per rating"), quasi-measure saat dirata-rata dengan kehati-hatian.

Exercise 2.2 — Deteksi masalah grain (Intermediate · 20 menit)

Objective: mengenali fan-out sebelum terjadi

Seorang analyst men-JOIN orders → order_items → products, lalu menghitung COUNT(DISTINCT order_id) untuk jumlah transaksi dan SUM(orders.shipping_fee) untuk total ongkir. Mana yang benar, mana yang salah, dan mengapa?

Answer key 2.2

COUNT(DISTINCT order_id) benar — DISTINCT menetralkan penggandaan akibat JOIN ke grain item. SUM(shipping_fee) salah — shipping_fee berada di grain order; setelah JOIN ke order_items, fee order berisi 3 item terjumlah 3x. Perbaikan: agregasi ongkir di tabel orders sebelum/terpisah dari JOIN, atau jumlahkan dari subquery ber-grain order. Kesalahan umum: mengira DISTINCT menyelesaikan semua masalah fan-out — DISTINCT hanya menolong COUNT, tidak menolong SUM.

Exercise 2.3 — Audit integrity (Advanced · 30 menit)

Objective: menerapkan enam dimensi pada kasus nyata

Tim marketing mengeluh: jumlah pelanggan di dashboard (2,4 juta) berbeda dari jumlah penerima email campaign (1,9 juta). Susun hipotesis per dimensi integrity yang bisa menjelaskan selisih 500 ribu, dan urutan pengecekannya.

Answer key 2.3

Hipotesis kuat: (1) Uniqueness — dashboard menghitung logical duplicates, email platform melakukan dedup by email; (2) Validity — email invalid/kosong tidak bisa dikirim (cek % email NULL/malformed); (3) Consistency — definisi berbeda: "pelanggan terdaftar" vs "pelanggan subscribed & contactable"; (4) Timeliness — snapshot diambil tanggal berbeda. Urutan cek: definisi dulu (paling sering), lalu dedup, lalu validitas email. Pelajaran: selisih angka antar sistem hampir selalu masalah definisi/duplikasi, bukan "data hilang".

Portfolio idea

★ PORTFOLIO TIP

Ambil satu dataset publik (mis. e-commerce Olist di Kaggle). Tulis dokumen 2 halaman: grain tiap tabel, relasi antar tabel, dan temuan audit enam dimensi. Ini artefak "data profiling" yang jarang dimiliki portofolio junior.

Further reading

Kimball & Ross, *The Data Warehouse Toolkit* bab 1 (grain) · DAMA-DMBOK: Data Quality dimensions.

Next chapter

Bab 3: kosakata ini dipakai untuk mulai berbicara dengan database — SQL dari SELECT pertama sampai JOIN dan agregasi.

Bab 3 — SQL Fundamental

Dari SELECT pertama sampai JOIN dan agregasi — dengan disiplin produksi sejak awal

Learning objectives	Menulis query SELECT-WHERE-GROUP BY-HAVING-ORDER BY dengan benar; memahami urutan eksekusi logis; menguasai semua jenis JOIN dan UNION; memakai CASE dan conditional aggregation untuk pivot analitis
Prerequisites	Bab 2 (grain, keys, NULL)
Estimasi waktu	Baca 2,5 jam · Praktik 6+ jam
Difficulty	● Beginner → ● Intermediate
Skills covered	SELECT, WHERE, GROUP BY, HAVING, ORDER BY, JOIN, UNION, CASE, aggregation
Business domain	Retail (TokoNusa)

Mengapa SQL — dan mengapa dikuasai sampai refleks

SQL adalah bahasa universal untuk bertanya kepada data. Empat puluh tahun berlalu, ribuan tool datang dan pergi, SQL tetap menjadi antarmuka utama setiap warehouse: PostgreSQL, BigQuery, Snowflake, Databricks — semuanya berbicara SQL.

Untuk analyst, SQL bukan skill opsional. Ia adalah perbedaan antara menunggu orang lain menyiapkan data dan mengambil sendiri jawaban dalam hitungan menit.

■ TECHNICAL NOTE — Dialek

Buku ini memakai SQL standar yang berjalan di PostgreSQL. Perbedaan dialek (BigQuery, Snowflake, MySQL) hanya di fungsi tanggal dan beberapa detail — konsepnya identik. Perbedaan penting akan ditandai saat muncul.

Anatomi query dan urutan eksekusi logis

Kesalahan pemahaman paling umum: mengira query dieksekusi dari atas ke bawah sesuai urutan penulisan. Kenyataannya, database mengeksekusi dalam urutan logis yang berbeda — dan banyak error serta kebingungan berasal dari sini.

URUTAN DITULIS	URUTAN DIEKSEKUSI (logis)
SELECT	1. FROM / JOIN ← ambil & gabung tabel
FROM / JOIN	2. WHERE ← filter baris mentah
WHERE	3. GROUP BY ← bentuk grup
GROUP BY	4. HAVING ← filter grup
HAVING	5. SELECT ← hitung & pilih kolom
ORDER BY	6. ORDER BY ← urutkan hasil
LIMIT	7. LIMIT ← potong hasil

Dua konsekuensi praktis yang wajib diingat:

- Alias kolom di SELECT belum ada saat WHERE dievaluasi — karena itu `WHERE net_revenue > 0` gagal jika `net_revenue` adalah alias (kecuali di beberapa dialek).

- WHERE menyaring *baris sebelum* agregasi; HAVING menyaring *grup setelah* agregasi. Keduanya bukan pilihan gaya — semantiknya berbeda.

SELECT dan WHERE — mengambil dan menyaring

Simple example

Business question: tampilkan order online yang selesai, terbaru dulu

```
SELECT
  order_id,
  customer_id,
  order_ts,
  status
FROM orders
WHERE channel = 'online'
  AND status = 'completed'
ORDER BY order_ts DESC
LIMIT 100;
```

Expected output: 100 baris, grain = 1 order, hanya online & completed. Perhatikan gaya penulisan: satu kolom per baris, keyword kapital, indentasi konsisten. Gaya ini bukan estetika — query akan dibaca ulang oleh orang lain (dan dirimu 6 bulan lagi).

Operator filter yang wajib dikuasai

Operator	Contoh	Catatan
=, !=, >, <, >=, <=	unit_price >= 50000	NULL tidak pernah lolos operator ini
IN	city IN ('Jakarta', 'Bandung')	Lebih bersih dari OR berantai
BETWEEN	order_ts BETWEEN '2026-07-01' AND '2026-07-31'	Inklusif dua sisi — hati-hati dengan timestamp (lihat Warning)
LIKE / ILIKE	product_name ILIKE '%kopi%'	% = apa saja; ILIKE = case-insensitive (PostgreSQL)
IS NULL / IS NOT NULL	customer_id IS NULL	Satu-satunya cara menguji NULL

⚠ WARNING — BETWEEN dan timestamp

`order_ts BETWEEN '2026-07-01' AND '2026-07-31'` pada kolom timestamp berhenti di *31 Juli 00:00:00* — hampir seluruh transaksi tanggal 31 hilang. Pola aman yang dipakai di produksi: `order_ts >= '2026-07-01' AND order_ts < '2026-08-01'`. Setengah-terbuka (inklusif-eksklusif) adalah standar profesional untuk rentang waktu.

Agregasi dan GROUP BY — dari baris ke ringkasan

Mengapa

Hampir semua pertanyaan bisnis adalah pertanyaan agregat: total, rata-rata, jumlah pelanggan. GROUP BY adalah mesin yang mengubah jutaan baris transaksi menjadi ringkasan yang bisa ditindaklanjuti — implementasi langsung dari "measure by dimension" di Bab 2.

Lima fungsi agregasi inti

Fungsi	Menghitung	Perlakuan NULL
COUNT(*)	Jumlah baris	Menghitung semua baris
COUNT(kolom)	Baris dengan kolom tidak NULL	NULL diabaikan
COUNT(DISTINCT k)	Nilai unik	NULL diabaikan
SUM / AVG	Total / rata-rata	NULL diabaikan — AVG bisa bias
MIN / MAX	Ekstrem	NULL diabaikan

Business example

Business question (Regional Manager): revenue & jumlah transaksi per region, Juli 2026

```
SELECT
  s.region,
  COUNT(DISTINCT o.order_id) AS total_orders,
  SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS net_revenue
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN stores s ON s.store_id = o.store_id
WHERE o.order_ts >= '2026-07-01'
      AND o.order_ts < '2026-08-01'
      AND o.status = 'completed'
GROUP BY s.region
ORDER BY net_revenue DESC;
```

Expected output

region	total_orders	net_revenue
Jawa Barat	312,441	48,220,510,000
Jakarta	287,113	45,101,330,000
Jawa Timur	198,674	30,455,870,000

Baris demi baris: JOIN ke order_items menurunkan grain ke item — karena itu jumlah transaksi memakai COUNT(DISTINCT order_id), bukan COUNT(*). Net revenue dihitung di grain item (tempat harga dan diskon hidup). Filter status membuang order batal. Ini penerapan langsung pelajaran grain Bab 2.

✗ COMMON MISTAKE — Kolom di SELECT tidak ada di GROUP BY

Gejala: error `column "s.store_name" must appear in the GROUP BY clause` — atau lebih buruk, di MySQL mode lama, query jalan dengan nilai semauanya.

Mengapa terjadi: setiap kolom non-agregat di SELECT wajib menjadi bagian definisi grup.

Perbaikan: aturan mekanis — semua kolom di SELECT harus (a) di dalam fungsi agregasi, atau (b) tercantum di GROUP BY. Tidak ada pengecualian.

HAVING — filter setelah agregasi

Business question: kategori mana yang revenue Juli-nya di atas 5 miliar?

```
SELECT
  p.category,
  SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS net_revenue
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
```

```
WHERE o.order_ts >= '2026-07-01' AND o.order_ts < '2026-08-01'
GROUP BY p.category
HAVING SUM(oi.quantity * oi.unit_price - oi.discount_amount) > 5000000000
ORDER BY net_revenue DESC;
```

Aturan pemilihan yang tidak pernah gagal: kondisi tentang *baris individual* (tanggal, status, channel) → WHERE. Kondisi tentang *hasil agregasi* (total > X, count ≥ N) → HAVING. Memfilter di WHERE apa yang seharusnya bisa difilter di WHERE juga lebih cepat — baris terbangun sebelum kerja agregasi.

JOIN — menggabungkan tabel

Mengapa

Data operasional sengaja dipecah ke banyak tabel (normalisasi) agar tidak redundan. Pertanyaan bisnis hampir selalu membutuhkan beberapa tabel sekaligus: "revenue per region" butuh order_items + orders + stores. JOIN adalah cara menyatukannya kembali.

Empat jenis JOIN dan kapan memakainya

INNER JOIN hanya yang cocok $A \cap B$ "order yang punya data customer"	LEFT JOIN semua kiri + pasangan (NULL jika tak ada) $A \bowtie B$ "SEMUA customer, termasuk yang belum pernah order"
RIGHT JOIN kebalikan LEFT (jarang; tulis ulang sebagai LEFT)	FULL OUTER JOIN semua dua sisi "rekonsiliasi dua sumber: apa yang hanya ada di salah satu?"

Jenis	Pakai ketika	Frekuensi di produksi
INNER	Hanya butuh baris yang punya pasangan	Sangat sering
LEFT	Sisi kiri wajib utuh (mis. semua customer, order-nya opsional)	Sangat sering
RIGHT	Hampir tidak pernah — tulis ulang sebagai LEFT agar terbaca	Jarang
FULL OUTER	Rekonsiliasi dua sumber data	Kadang, untuk audit

Business example — pertanyaan yang hanya bisa dijawab LEFT JOIN

Business question (CRM lead): siapa pelanggan terdaftar yang belum pernah bertransaksi? (target campaign reaktivasi)

```
SELECT
  c.customer_id,
  c.full_name,
  c.join_date
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;      -- tak ada pasangan = belum pernah order
```

Pola *LEFT JOIN ... WHERE right.key IS NULL* ("anti-join") adalah salah satu pola paling berguna di analytics: menemukan yang *tidak* melakukan sesuatu. INNER JOIN tidak akan pernah bisa menjawab pertanyaan ini — baris tanpa pasangan justru dibuangnya.

✗ COMMON MISTAKE — Filter sisi kanan di WHERE membunuh LEFT JOIN

Gejala: LEFT JOIN ditulis, tetapi hasil hanya berisi baris yang punya pasangan — LEFT-nya "hilang".

```
-- SALAH: baris customer tanpa order punya o.status = NULL,  
-- dan NULL = 'completed' bukan TRUE → baris terbuang  
LEFT JOIN orders o ON o.customer_id = c.customer_id  
WHERE o.status = 'completed'  
  
-- BENAR: kondisi sisi kanan dipindah ke ON  
LEFT JOIN orders o ON o.customer_id = c.customer_id  
AND o.status = 'completed'
```

Aturan: pada LEFT JOIN, filter untuk tabel kanan diletakkan di ON; filter untuk tabel kiri di WHERE.

▲ PERFORMANCE TIP

JOIN pada kolom ber-index (PK/FK) cepat; JOIN pada hasil fungsi (`ON UPPER(a.email) = UPPER(b.email)`) memaksa full scan. Jika harus mencocokkan dengan transformasi, bersihkan datanya dulu di tahap preparation (Bab 6), jangan di JOIN.

UNION — menumpuk hasil, bukan menggabung kolom

JOIN menggabungkan tabel ke samping (menambah kolom). UNION menumpuk hasil ke bawah (menambah baris). Pakai saat dua sumber ber-struktur sama harus jadi satu — misalnya penjualan sistem lama dan sistem baru pasca-migrasi.

```
SELECT order_id, order_ts, net_amount, 'legacy' AS source_system  
FROM legacy_sales  
WHERE order_ts < '2026-01-01'  
UNION ALL  
SELECT order_id, order_ts, net_amount, 'new'  
FROM orders_flat  
WHERE order_ts >= '2026-01-01';
```

- **UNION ALL** menumpuk apa adanya — cepat. **UNION** (tanpa ALL) membuang duplikat antar hasil — mahal, dan sering menghapus baris yang sah (dua transaksi identik yang memang berbeda).
- Default profesional: UNION ALL, kecuali ada alasan eksplisit deduplikasi.
- Kolom sumber (`source_system`) adalah kebiasaan senior: saat angka aneh, langsung ketahuan sumbernya.

CASE — logika kondisional di dalam query

Mengapa

CASE mengubah nilai mentah menjadi kategori bisnis: rentang harga → "budget/premium", tanggal → "weekday/weekend", status gabungan → satu label bersih. Ia adalah alat utama menerjemahkan aturan bisnis ke dalam SQL.

Simple example

```
SELECT  
    product_name,  
    retail_price,
```



```

CASE
  WHEN retail_price < 50000 THEN 'Budget'
  WHEN retail_price < 250000 THEN 'Mid-range'
  ELSE 'Premium'
END AS price_tier
FROM products;

```

CASE dievaluasi berurutan dan berhenti di kondisi pertama yang cocok — karena itu batas bawah tidak perlu diulang. Selalu sertakan ELSE; tanpa ELSE, baris yang tidak cocok menjadi NULL diam-diam.

Conditional aggregation — teknik pivot paling berguna di SQL

Mengapa

Gabungan CASE + agregasi memungkinkan satu query menghasilkan beberapa metric bersyarat berdampingan — bentuk tabel yang paling sering diminta manajemen ("online vs offline per region dalam satu tabel").

Business example

Business question: revenue online vs offline per region, plus % online — satu tabel

```

SELECT
  s.region,
  SUM(CASE WHEN o.channel = 'online'
    THEN oi.quantity * oi.unit_price - oi.discount_amount ELSE 0 END) AS online_rev,
  SUM(CASE WHEN o.channel = 'offline'
    THEN oi.quantity * oi.unit_price - oi.discount_amount ELSE 0 END) AS offline_rev,
  ROUND(
    100.0 * SUM(CASE WHEN o.channel = 'online'
      THEN oi.quantity * oi.unit_price - oi.discount_amount ELSE 0 END)
    / NULLIF(SUM(oi.quantity * oi.unit_price - oi.discount_amount), 0)
    , 1) AS online_pct
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN stores s      ON s.store_id = o.store_id
WHERE o.order_ts >= '2026-07-01' AND o.order_ts < '2026-08-01'
  AND o.status = 'completed'
GROUP BY s.region;

```

Tiga detail produksi di query ini: (1) `ELSE 0` menjaga SUM tetap terdefinisi; (2) `100.0` memaksa pembagian desimal — `100 * a / b` pada integer bisa membulat ke 0; (3) `NULLIF(x, 0)` mengubah pembagi 0 menjadi NULL alih-alih error division-by-zero. Tiga kebiasaan kecil ini membedakan query latihan dari query produksi.

✓ BEST PRACTICE — Standar penulisan query

- ✓ Keyword kapital, identifier lowercase_snake_case
- ✓ Satu kolom per baris pada SELECT panjang
- ✓ Alias tabel pendek dan konsisten (o, oi, c, s, p)
- ✓ Komentar untuk keputusan bisnis, bukan untuk hal yang jelas
- ✓ Filter tanggal setengah-terbuka
- ✓ NULLIF pada setiap pembagian
- ✓ Simpan query dengan nama yang menjelaskan pertanyaannya, bukan "query_final_v2.sql".

Membuktikan grain dengan query — ritual dari Bab 2

Sekarang kamu punya alat untuk menjalankan ritual pembuktian grain:

```
-- Jika tabel benar ber-grain 1 baris = 1 order, dua angka ini identik:
SELECT
    COUNT(*)                AS total_rows,
    COUNT(DISTINCT order_id) AS distinct_orders
FROM orders;
-- total_rows > distinct_orders → ada duplikat / grain bukan order. Selidiki.
```

Checklist query fundamental

- ✓ Grain hasil query diketahui dan disebutkan di komentar/dokumentasi
- ✓ Filter tanggal memakai pola setengah-terbuka ($\geq$ dan $\leq$)
- ✓ Status transaksi difilter eksplisit (completed vs cancelled/refunded)
- ✓ COUNT memakai DISTINCT saat grain sudah diturunkan oleh JOIN
- ✓ Filter tabel kanan LEFT JOIN berada di ON, bukan WHERE
- ✓ Setiap pembagian dilindungi NULLIF; rasio memakai angka desimal
- ✓ UNION ALL dipakai kecuali deduplikasi memang diinginkan
- ✓ Total hasil direkonsiliasi dengan angka referensi sebelum dikirim

Ending Card — Bab 3

What you learned

Urutan eksekusi logis; SELECT-WHERE-GROUP BY-HAVING-ORDER BY; empat jenis JOIN + anti-join; UNION vs UNION ALL; CASE; conditional aggregation dengan proteksi produksi.

Skills mastered

Menerjemahkan pertanyaan bisnis → query multi-tabel · pivot analitis satu-query · disiplin gaya penulisan produksi.

Real business value

Kemampuan menjawab mayoritas pertanyaan ad-hoc manajemen (revenue by X, top N, siapa-yang-tidak) dalam hitungan menit, dengan angka yang bisa dipertanggungjawabkan.

Interview readiness

● INTERVIEW QUESTIONS

1. "Bedanya WHERE dan HAVING?" — jawab lewat urutan eksekusi, beri contoh.
2. "Bedanya INNER dan LEFT JOIN? Kapan LEFT wajib?" — contoh kuat: analisis pelanggan yang belum pernah order.
3. "Kenapa hasil LEFT JOIN-mu kehilangan baris kiri?" — filter sisi kanan di WHERE; jelaskan perbaikan ke ON.
4. Live coding paling umum: "revenue per kategori per bulan, hanya kategori dengan revenue > X" — persis pola GROUP BY + HAVING bab ini.

Exercises

Exercise 3.1 (Beginner · 15 menit)

Dataset: skema TokoNusa · Expected output: 1 tabel kecil

Tulis query: 10 produk dengan quantity terjual terbanyak selama Juni 2026, tampilkan nama produk, kategori, dan total quantity. Hanya order completed.

Answer key 3.1

```
SELECT p.product_name, p.category, SUM(oi.quantity) AS total_qty
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN products p ON p.product_id = oi.product_id
WHERE o.order_ts >= '2026-06-01' AND o.order_ts < '2026-07-01'
      AND o.status = 'completed'
GROUP BY p.product_name, p.category
ORDER BY total_qty DESC
LIMIT 10;
```

Common mistake: lupa filter status, atau memakai BETWEEN '2026-06-30' yang memotong hari terakhir.

Exercise 3.2 (Intermediate · 25 menit)

Objective: anti-join + agregasi

Marketing ingin daftar pelanggan segment 'Gold' yang TIDAK bertransaksi sama sekali di Juli 2026 (kandidat winback). Tampilkan customer_id, full_name, dan tanggal transaksi terakhir mereka (kapan pun itu).

Answer key 3.2

```
SELECT c.customer_id, c.full_name, MAX(o_all.order_ts) AS last_order_ts
FROM customers c
LEFT JOIN orders o_jul
    ON o_jul.customer_id = c.customer_id
    AND o_jul.order_ts >= '2026-07-01' AND o_jul.order_ts < '2026-08-01'
    AND o_jul.status = 'completed'
LEFT JOIN orders o_all
    ON o_all.customer_id = c.customer_id AND o_all.status = 'completed'
WHERE c.segment = 'Gold'
    AND o_jul.order_id IS NULL
GROUP BY c.customer_id, c.full_name;
```

Poin penilaian: filter Juli berada di ON (bukan WHERE) agar anti-join bekerja; transaksi terakhir dihitung dari JOIN kedua tanpa batas waktu. Alternatif yang juga benar: NOT EXISTS dengan subquery (Bab 4) — lebih terbaca untuk banyak orang.

Exercise 3.3 (Advanced · 35 menit)

Objective: conditional aggregation produksi

Buat satu query "channel mix report" per kategori produk untuk Q2 2026: revenue online, revenue offline, % online, AOV (average order value) keseluruhan. Lindungi semua pembagian.

Answer key 3.3 (kerangka inti)

```
SELECT p.category,
       SUM(CASE WHEN o.channel='online' THEN oi.quantity*oi.unit_price-oi.discount_amount
       ELSE 0 END) AS online_rev,
       SUM(CASE WHEN o.channel='offline' THEN oi.quantity*oi.unit_price-oi.discount_amount
       ELSE 0 END) AS offline_rev,
       ROUND(100.0*SUM(CASE WHEN o.channel='online' THEN oi.quantity*oi.unit_price-
       oi.discount_amount ELSE 0 END)
       /NULLIF(SUM(oi.quantity*oi.unit_price-oi.discount_amount),0),1) AS online_pct,
       ROUND(SUM(oi.quantity*oi.unit_price-oi.discount_amount)
       /NULLIF(COUNT(DISTINCT o.order_id),0),0) AS aov
FROM orders o
JOIN order_items oi ON oi.order_id=o.order_id
JOIN products p ON p.product_id=oi.product_id
WHERE o.order_ts >= '2026-04-01' AND o.order_ts < '2026-07-01'
AND o.status='completed'
GROUP BY p.category;
```

Catatan penting: AOV per kategori di sini adalah aproksimasi — satu order lintas kategori terhitung penuh di tiap kategorinya. Menyebutkan keterbatasan ini adalah tanda kandidat yang paham grain; solusi presisinya (alokasi per item) dibahas di Bab 4.

Portfolio idea

★ PORTFOLIO TIP

Bangun "SQL case study" di GitHub: 10 pertanyaan bisnis atas dataset publik, masing-masing dengan business question → query → hasil → interpretasi 2 kalimat. Format ini meniru cara kerja sesungguhnya dan jauh lebih meyakinkan daripada sertifikat.

Further reading

PostgreSQL Tutorial (postgresqltutorial.com) · Mode SQL Tutorial · *SQL for Data Analysis* (Cathy Tanimura), bab 1–3.

Next chapter

Bab 4 naik ke SQL tingkat produksi: subquery, CTE untuk analisis bertingkat, window function untuk ranking/tren, dan manipulasi tanggal — kelompok skill yang memisahkan junior dari mid-level.

Bab 4 — SQL Tingkat Produksi

Subquery, CTE, window function, dan date logic — skill yang memisahkan junior dari mid-level

Learning objectives	Menstrukturkan analisis bertingkat dengan CTE; menguasai window function untuk ranking, tren, dan perbandingan; manipulasi tanggal untuk analisis periode; membangun query produksi yang terbaca dan teruji
Prerequisites	Bab 3 dikuasai penuh
Estimasi waktu	Baca 3 jam · Praktik 8+ jam
Difficulty	● Intermediate → ● Advanced
Skills covered	Subquery, CTE, window functions, date manipulation, query patterns produksi
Business domain	Retail (TokoNusa)

Mengapa level ini yang paling menentukan karier

Pertanyaan bisnis yang bernilai tinggi hampir selalu bertingkat: "bandingkan tiap pelanggan dengan rata-rata segmennya", "urutkan produk terlaris per region", "hitung pertumbuhan bulan-ke-bulan". Pertanyaan seperti ini tidak bisa dijawab satu lapis SELECT.

Tiga alat bab ini — subquery, CTE, window function — adalah cara SQL menyusun penalaran bertingkat. Di interview data analyst, bagian tersulit live coding hampir selalu berasal dari sini.

Subquery — query di dalam query

Tiga posisi subquery

Posisi	Bentuk	Contoh penggunaan
Di WHERE (scalar)	Menghasilkan 1 nilai	Produk dengan harga di atas rata-rata
Di WHERE (IN / EXISTS)	Menghasilkan daftar	Order dari pelanggan Gold
Di FROM (derived table)	Menghasilkan tabel	Agregasi dua tahap

Simple example — scalar subquery

Business question: produk apa yang harganya di atas rata-rata katalog?

```
SELECT product_name, retail_price
FROM products
WHERE retail_price > (SELECT AVG(retail_price) FROM products);
```

EXISTS vs IN

```
-- Order milik pelanggan Gold — dua cara setara secara hasil:
SELECT * FROM orders o
WHERE o.customer_id IN (SELECT customer_id FROM customers WHERE segment = 'Gold');
```

```
SELECT * FROM orders o
WHERE EXISTS (SELECT 1 FROM customers c
              WHERE c.customer_id = o.customer_id AND c.segment = 'Gold');
```

⚠ WARNING — NOT IN dan NULL

`NOT IN (subquery)` mengembalikan *not baris* jika subquery mengandung satu saja NULL — konsekuensi three-valued logic Bab 2, dan sumber bug senyap yang terkenal. Standar produksi: pakai `NOT EXISTS`, yang kebal terhadap NULL.

CTE — menstrukturkan analisis seperti paragraf

Mengapa

Subquery bersarang tiga lapis tidak bisa dibaca, tidak bisa direview, dan tidak bisa di-debug. CTE (Common Table Expression, klausa `WITH`) memecah analisis menjadi langkah bernama yang dibaca dari atas ke bawah — seperti paragraf dalam esai.

Business example — analisis dua tahap

Business question (CF0): berapa rata-rata belanja bulanan pelanggan per segment? (bukan rata-rata transaksi — rata-rata PELANGGAN)

```
WITH monthly_spend AS (
  -- Tahap 1: total belanja tiap pelanggan tiap bulan (grain: customer x bulan)
  SELECT
    o.customer_id,
    DATE_TRUNC('month', o.order_ts) AS order_month,
    SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS spend
  FROM orders o
  JOIN order_items oi ON oi.order_id = o.order_id
  WHERE o.status = 'completed'
    AND o.order_ts >= '2026-01-01' AND o.order_ts < '2026-07-01'
  GROUP BY o.customer_id, DATE_TRUNC('month', o.order_ts)
)
-- Tahap 2: rata-ratakan per segment (grain: segment)
SELECT
  c.segment,
  ROUND(AVG(ms.spend), 0) AS avg_monthly_spend_per_customer
FROM monthly_spend ms
JOIN customers c ON c.customer_id = ms.customer_id
GROUP BY c.segment
ORDER BY avg_monthly_spend_per_customer DESC;
```

Mengapa harus dua tahap: rata-rata langsung atas transaksi memberi bobot lebih besar ke pelanggan yang sering belanja — menjawab pertanyaan yang berbeda. Perbedaan "average of averages" vs "average keseluruhan" adalah tema yang akan terus muncul sampai Semester 2 (Simpson's paradox).

✓ BEST PRACTICE — Disiplin CTE

✓ Satu CTE = satu langkah logis dengan nama deskriptif (`monthly_spend`, bukan `cte1`) ✓ Tulis grain tiap CTE sebagai komentar ✓ Debug dengan mengganti query akhir menjadi `SELECT * FROM nama_cte LIMIT 20` per tahap ✓ Lebih dari ±5 CTE panjang: pertimbangkan memecah menjadi staging table di pipeline (Bab 5).

Window function — agregasi tanpa menghancurkan baris

Mengapa ini alat terpenting bab ini

GROUP BY meringkas dan *menghilangkan* baris individual. Window function menghitung agregat, ranking, atau perbandingan *sambil mempertahankan* setiap baris. Semua analisis "per baris terhadap konteksnya" — kontribusi item terhadap total, ranking dalam grup, perbandingan dengan bulan lalu — hidup di sini.

Anatomi

```
fungsi(...) OVER (  
  PARTITION BY kolom  -- reset perhitungan per grup (opsional)  
  ORDER BY kolom      -- urutan dalam grup (untuk ranking/lag/running)  
)
```

Keluarga fungsi yang wajib dikuasai

Fungsi	Menghasilkan	Kasus khas
ROW_NUMBER()	1,2,3,... unik per partisi	Ambil "1 baris terbaru per pelanggan"; deduplikasi
RANK() / DENSE_RANK()	Ranking dengan seri (skip / tanpa skip)	Top-N produk per region
LAG() / LEAD()	Nilai baris sebelum/sesudah	Growth MoM; jeda antar pembelian
SUM/AVG(...) OVER	Agregat per partisi di tiap baris	% kontribusi terhadap total grup
SUM(...) OVER (ORDER BY)	Running total	Kumulatif pencapaian target
AVG(...) OVER (... ROWS 6 PRECEDING)	Moving average	Tren harian 7 hari

Pola 1 — Top-N per grup (pola interview paling sering)

Business question: 3 produk ber-revenue tertinggi di TIAP region, Juli 2026

```
WITH product_region AS (  
  SELECT  
    s.region, p.product_name,  
    SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS revenue  
  FROM order_items oi  
  JOIN orders o    ON o.order_id = oi.order_id  
  JOIN stores s    ON s.store_id = o.store_id  
  JOIN products p  ON p.product_id = oi.product_id  
  WHERE o.order_ts >= '2026-07-01' AND o.order_ts < '2026-08-01'  
        AND o.status = 'completed'  
  GROUP BY s.region, p.product_name  
)  
,  
ranked AS (  
  SELECT *,  
    DENSE_RANK() OVER (PARTITION BY region ORDER BY revenue DESC) AS rnk  
  FROM product_region  
)  
SELECT region, rnk, product_name, revenue  
FROM ranked  
WHERE rnk <= 3  
ORDER BY region, rnk;
```


■ TECHNICAL NOTE — Kenapa perlu CTE kedua

Window function dievaluasi di tahap SELECT — setelah WHERE. Karena itu `WHERE rnk <= 3` tidak bisa ditulis di query yang sama tempat `rnk` didefinisikan; ia harus dibungkus CTE/subquery dulu. Ini pertanyaan jebakan interview yang sangat umum.

Pola 2 — Growth month-over-month dengan LAG

Business question (CEO deck): pertumbuhan revenue bulanan 2026

```
WITH monthly AS (  
  SELECT  
    DATE_TRUNC('month', o.order_ts) AS month,  
    SUM(oi.quantity * oi.unit_price - oi.discount_amount) AS revenue  
  FROM orders o  
  JOIN order_items oi ON oi.order_id = o.order_id  
  WHERE o.status = 'completed' AND o.order_ts >= '2026-01-01'  
  GROUP BY 1  
)  
SELECT  
  month,  
  revenue,  
  LAG(revenue) OVER (ORDER BY month) AS prev_month,  
  ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))  
    / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 1) AS mom_growth_pct  
FROM monthly  
ORDER BY month;
```

Expected output

month	revenue	prev_month	mom_growth_pct
2026-01-01	118,240,000,000	NULL	NULL
2026-02-01	109,880,000,000	118,240,000,000	-7.1
2026-03-01	131,470,000,000	109,880,000,000	19.6

◆ BUSINESS INSIGHT — Sebelum melaporkan growth

MoM -7,1% di Februari belum tentu masalah: Februari punya 3 hari lebih sedikit (−10% hari kalender) dan efek musiman pasca-Januari. Analyst yang matang melaporkan growth bersama konteksnya: jumlah hari, seasonality, dan event (payday, kampanye, hari raya). Pengujian formal seasonality dipelajari di Semester 2.

Pola 3 — Kontribusi terhadap total (window agregat)

```
SELECT  
  category,  
  revenue,  
  ROUND(100.0 * revenue / SUM(revenue) OVER (), 1) AS pct_of_total  
FROM category_revenue -- CTE agregasi seperti sebelumnya  
ORDER BY revenue DESC;
```

Pola 4 — Deduplikasi dengan ROW_NUMBER

Ambil hanya baris terbaru per pelanggan dari tabel yang mengandung duplikat historis

```
WITH ranked AS (  
  SELECT *,
```

```

ROW_NUMBER() OVER (
  PARTITION BY customer_id
  ORDER BY updated_at DESC
) AS rn
FROM customer_snapshots
)
SELECT * FROM ranked WHERE rn = 1;

```

Pola ini adalah alat deduplikasi standar industri — akan dipakai lagi di Bab 6 dan di hampir setiap pipeline yang pernah kamu sentuh.

Manipulasi tanggal — bahasa waktu dalam bisnis

Hampir semua analisis punya dimensi waktu. Fungsi inti (sintaks PostgreSQL, padanan dialek lain serupa):

Kebutuhan	Fungsi	Contoh
Potong ke periode	<code>DATE_TRUNC('month', ts)</code>	2026-07-19 14:32 → 2026-07-01
Ambil komponen	<code>EXTRACT(DOW/HOUR/... FROM ts)</code>	Hari-minggu, jam sibuk
Selisih	<code>ts_a - ts_b / AGE()</code>	Lead time pengiriman
Aritmetika	<code>ts + INTERVAL '30 days'</code>	Window retensi D+30
Periode relatif	<code>CURRENT_DATE - INTERVAL '90 days'</code>	"90 hari terakhir" dinamis

■ TECHNICAL NOTE — Dialek

BigQuery: `DATE_TRUNC(ts, MONTH)`, `DATE_DIFF/TIMESTAMP_DIFF`. Snowflake: `DATE_TRUNC('MONTH', ts)`, `DATEDIFF`. MySQL: `DATE_FORMAT(ts, '%Y-%m-01')`. Konsepnya identik; hafalkan konsep, cari sintaks.

⚠ WARNING — Timezone

Warehouse umumnya menyimpan timestamp dalam UTC. "Penjualan hari Senin" dalam UTC bisa berbeda dari Senin WIB (UTC+7) — transaksi Senin 00:00-06:59 WIB tercatat sebagai Minggu UTC. Untuk laporan bisnis Indonesia, konversi dulu: `order_ts AT TIME ZONE 'Asia/Jakarta'`. Kesalahan timezone adalah penyebab klasik "angka dashboard beda dengan laporan toko".

Kebiasaan produksi: validasi query bertingkat

Query kompleks tidak divalidasi dengan menatap kodenya, tetapi dengan protokol:

1. **Uji per tahap.** Jalankan setiap CTE sendiri, periksa grain dan sampel barisnya.
2. **Rekonsiliasi total.** Jumlah akhir harus cocok dengan angka referensi yang sudah dipercaya (laporan finance, dashboard resmi). Selisih > 0,5% = selidiki sebelum kirim.
3. **Uji kasus tepi.** Pelanggan tanpa order, bulan tanpa transaksi, pembagi nol — apakah query tetap benar?
4. **Cek fan-out.** Bandingkan SUM measure sebelum dan sesudah setiap JOIN baru.

▲ PERFORMANCE TIP — Membaca sebelum mengoptimasi

Gunakan `EXPLAIN` untuk melihat rencana eksekusi. Tiga pemenang cepat yang menyelesaikan mayoritas masalah: (1) filter tanggal/partisi sedini mungkin, (2) jangan bungkus kolom filter dalam fungsi (`WHERE DATE(ts) = ...` mematikan index — pakai rentang), (3) `SELECT` hanya kolom yang dipakai, terutama di warehouse kolumnar (BigQuery/Snowflake) di mana biaya = kolom yang dipindai.

Checklist SQL produksi

- ✓ Analisis bertingkat memakai CTE bernama deskriptif, grain tertulis
- ✓ `NOT EXISTS` dipakai alih-alih `NOT IN` pada subquery nullable
- ✓ Filter atas hasil window function dibungkus CTE terpisah
- ✓ Growth/perbandingan dilaporkan dengan konteks (hari kalender, seasonality)
- ✓ Timezone dikonversi eksplisit untuk laporan lokal
- ✓ Total direkonsiliasi dengan sumber tepercaya sebelum dikirim
- ✓ Fan-out dicek setiap menambah `JOIN`

Ending Card — Bab 4

What you learned

Subquery tiga posisi; jebakan NOT IN + NULL; CTE untuk analisis bertingkat; empat pola window function inti (top-N, LAG growth, kontribusi, dedup); date logic dan timezone; protokol validasi query.

Skills mastered

Analisis multi-tahap yang terbaca dan teruji · pola-pola interview tersulit · disiplin validasi produksi.

Real business value

Mampu menjawab pertanyaan analitis kompleks (growth, ranking, retensi, kontribusi) tanpa menunggu data engineer — dengan angka yang tahan diaudit.

Interview readiness

● INTERVIEW QUESTIONS

1. "Bedanya ROW_NUMBER, RANK, DENSE_RANK?" — jawab dengan contoh nilai seri.
2. "Kenapa tidak bisa WHERE di kolom window function?" — urutan eksekusi; solusi CTE.
3. "Top 3 produk per kategori" — pola 1 bab ini, live coding paling umum se-industri.
4. "Hitung MoM growth" — pola 2; nilai plus jika menyebut konteks jumlah hari/seasonality.
5. "NOT IN mengembalikan 0 baris, kenapa?" — NULL di subquery; jawab NOT EXISTS.

Exercises

Exercise 4.1 (Intermediate · 25 menit)

Objective: window agregat + kontribusi

Buat laporan per toko untuk Juli 2026: revenue toko, revenue region-nya, dan % kontribusi toko terhadap region. Urutkan per region, kontribusi terbesar dulu.

Answer key 4.1 (inti)

```
WITH store_rev AS (
  SELECT s.store_id, s.store_name, s.region,
         SUM(oi.quantity*oi.unit_price-oi.discount_amount) AS revenue
  FROM orders o
  JOIN order_items oi ON oi.order_id=o.order_id
  JOIN stores s ON s.store_id=o.store_id
  WHERE o.order_ts >= '2026-07-01' AND o.order_ts < '2026-08-01'
         AND o.status='completed'
  GROUP BY 1,2,3
)
SELECT region, store_name, revenue,
       SUM(revenue) OVER (PARTITION BY region) AS region_revenue,
       ROUND(100.0*revenue/NULLIF(SUM(revenue) OVER (PARTITION BY region),0),1) AS
pct_region
FROM store_rev
ORDER BY region, pct_region DESC;
```

Exercise 4.2 (Advanced · 40 menit)

Objective: LAG lintas entitas

Hitung median jeda hari antar pembelian per segment (waktu dari satu order pelanggan ke order berikutnya). Petunjuk: LAG per customer, lalu agregasi per segment.

Answer key 4.2 (inti)

```
WITH gaps AS (
  SELECT o.customer_id,
         o.order_ts - LAG(o.order_ts) OVER (
           PARTITION BY o.customer_id ORDER BY o.order_ts) AS gap
  FROM orders o
  WHERE o.status = 'completed'
)
SELECT c.segment,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY EXTRACT(EPOCH FROM gap)/86400) AS
median_gap_days
FROM gaps g
JOIN customers c ON c.customer_id = g.customer_id
WHERE g.gap IS NOT NULL      -- order pertama tiap pelanggan tak punya jeda
GROUP BY c.segment;
```

Poin penilaian: PARTITION BY customer (bukan global), membuang baris pertama (gap NULL), dan memilih median bukan mean — distribusi jeda pembelian sangat skewed (alasan diformalkan di Semester 2). Common mistake: memakai AVG dan mendapat angka yang didistorsi pelanggan pasif.

Exercise 4.3 — Production challenge (Advanced · 60 menit)

Objective: menggabungkan seluruh bab

Bangun satu query "monthly customer health report" 2026: per bulan — jumlah pelanggan aktif, pelanggan baru (order pertama di bulan itu), pelanggan kembali, revenue, dan MoM growth revenue. Definisikan sendiri istilah yang ambigu, tulis asumsinya.

Answer key 4.3 (kerangka + poin kunci)

Struktur kuat: CTE-1 order bulanan per customer; CTE-2 first_order_month per customer (MIN DATE_TRUNC); CTE-3 agregasi bulanan dengan conditional aggregation — new = COUNT DISTINCT customer di mana bulan = first_order_month, returning = aktif — new; query akhir menambah LAG untuk growth. Poin kunci penilaian: definisi "aktif" dinyatakan (≥ 1 order completed); new vs returning eksklusif dan totalnya = aktif; growth dilindungi NULLIF. Ini miniatur capstone Bab 8.

Portfolio idea

★ PORTFOLIO TIP

Tambahkan ke case study GitHub-mu satu file "advanced_sql.sql" berisi 5 pola bab ini atas dataset publik, masing-masing dengan komentar business question dan interpretasi hasil. Sertakan satu contoh validasi (rekonsiliasi total) — hampir tidak ada kandidat junior yang menunjukkan kebiasaan validasi.

Further reading

PostgreSQL docs: [Window Functions](#) · *SQL for Data Analysis* (Tanimura) bab window & time series · [LeetCode/StrataScratch SQL medium-hard](#) untuk latihan pola.

Next chapter

Bab 5 pindah dari menulis query ke memahami rumah tempat data tinggal: OLTP vs OLAP, star schema, ETL/ELT, dan pipeline mindset yang membuat angkamu konsisten setiap hari.

Bab 5 — Arsitektur Data & Pipeline

OLTP, OLAP, star schema, ETL/ELT — dan pipeline mindset yang membuat angka konsisten setiap hari

Learning objectives	Membedakan OLTP vs OLAP dan konsekuensinya bagi analyst; membaca dan mendesain star schema; memahami ETL vs ELT dan lapisan warehouse modern; mengadopsi pipeline mindset: transformasi terversioning, teruji, terdokumentasi
Prerequisites	Bab 2-4
Estimasi waktu	Baca 2 jam · Praktik 3 jam
Difficulty	● Intermediate
Skills covered	OLTP/OLAP, data warehouse, star schema, fact & dimension, ETL/ELT, pipeline & data testing
Business domain	Retail (TokoNusa)

Mengapa analyst harus paham arsitektur

Analyst yang tidak paham arsitektur memperlakukan warehouse sebagai kotak hitam: query kadang lambat, angka kadang berubah, dan ia tidak tahu mengapa. Analyst yang paham arsitektur tahu tabel mana yang tepat untuk pertanyaan apa, mengapa angka dashboard bisa berbeda dari sistem kasir, dan bagaimana memperbaikinya di akar.

Di tim modern, batas analyst dan analytics engineer semakin kabur. Analyst yang bisa membangun transformasi yang teruji bernilai jauh lebih tinggi — dan bab ini adalah fondasinya.

OLTP vs OLAP — dua dunia dengan tujuan berlawanan

Mengapa dipisah

Sistem kasir TokoNusa harus mencatat transaksi dalam milidetik, ribuan kali per menit. Analisis revenue setahun harus memindai ratusan juta baris. Dua beban kerja ini saling membunuh jika berada di database yang sama: query analitik besar membuat kasir macet. Karena itu keduanya dipisah.

	OLTP (operasional)	OLAP (analitik)
Tujuan	Menjalankan bisnis: catat order, update stok	Memahami bisnis: agregasi, tren, laporan
Pola akses	Banyak transaksi kecil (1 baris)	Sedikit query besar (jutaan baris)
Desain	Normalized (minim redundansi)	Denormalized (dioptimalkan untuk baca)
Penyimpanan	Row-oriented (PostgreSQL, MySQL)	Column-oriented (BigQuery, Snowflake, Redshift)
Data	Kondisi saat ini	Sejarah bertahun-tahun
Pengguna	Aplikasi	Analyst, BI, data scientist

⚠ WARNING — Jangan analisis di database produksi

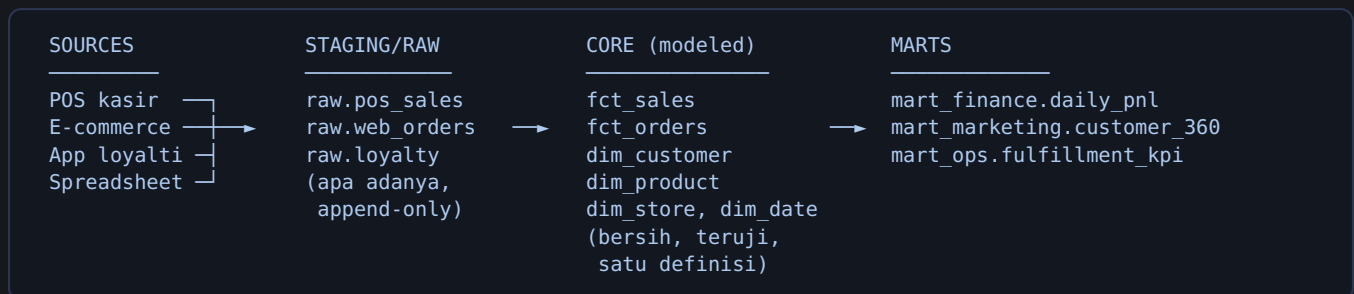
Menjalankan query analitik berat langsung di OLTP produksi dapat memperlambat aplikasi yang dipakai pelanggan — kesalahan yang bisa membuat analyst dipanggil engineering lead di hari pertamanya. Selalu bekerja di replica atau warehouse. Jika perusahaanmu belum punya, itu masalah pertama yang layak kamu bantu selesaikan.

■ TECHNICAL NOTE — Kenapa kolumnar cepat untuk analitik

Row store menyimpan baris utuh berurutan — bagus untuk "ambil order #123". Column store menyimpan tiap kolom berurutan — "SUM(revenue) setahun" hanya membaca kolom revenue, bukan seluruh tabel. Kompresi per kolom juga jauh lebih efektif. Konsekuensi praktis: di warehouse kolumnar, `SELECT *` membayar semua kolom; pilih kolom = hemat biaya dan waktu.

Data warehouse dan lapisan-lapisannya

Warehouse modern disusun berlapis. Memahami lapisan = tahu di mana harus bekerja:



- **Raw/staging:** salinan sumber apa adanya. Tidak pernah diedit — ini "bukti forensik" untuk debugging.
- **Core:** data yang sudah dimodelkan, dibersihkan, dan diuji. Di sinilah definisi metric hidup — *single source of truth*.
- **Marts:** potongan siap pakai per fungsi bisnis. Dashboard membaca dari sini.

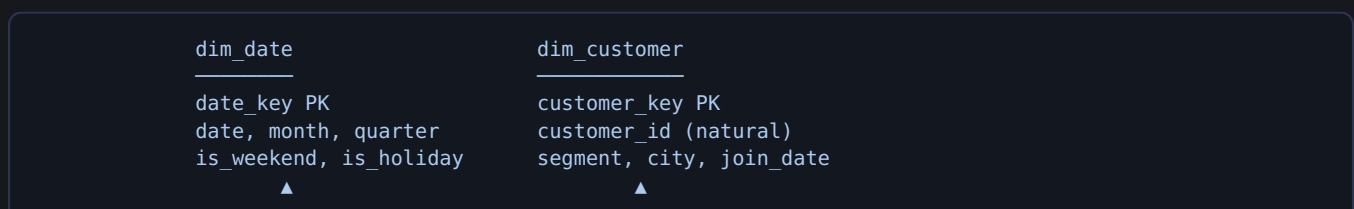
◆ BUSINESS INSIGHT — Single source of truth

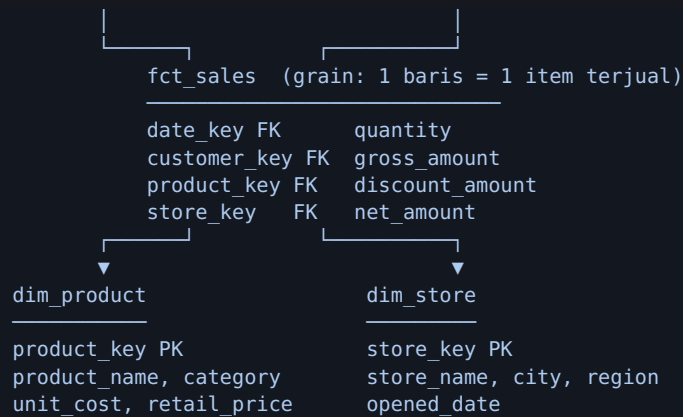
Ingat pelajaran konsistensi Bab 2: dua angka revenue yang berbeda menghancurkan kepercayaan. Arsitektur berlapis adalah solusinya — semua laporan menghitung dari core yang sama, dengan satu definisi `net_revenue`. Perusahaan yang metric-nya dihitung ulang di setiap dashboard akan selamanya rapat tentang "angka siapa yang benar".

Star schema — desain standar layer core

Mengapa bentuk bintang

Star schema menempatkan **fact table** (peristiwa terukur, grain sekecil mungkin) di tengah, dikelilingi **dimension table** (konteks deskriptif). Bentuk ini membuat query analitik menjadi pola yang selalu sama: JOIN fact ke dimension, filter di dimension, agregasi measure di fact — persis pola yang kamu latih di Bab 3–4.





Aturan desain yang perlu dipegang

- **Grain dideklarasikan dulu.** Kalimat pertama desain fact table: "satu baris = satu ____". Semua kolom harus konsisten dengan grain itu.
- **Fact berisi FK + measure numerik.** Teks deskriptif tinggal di dimension.
- **dim_date selalu ada.** Tabel kalender dengan flag is_weekend, is_holiday, fiscal_period — membuat analisis musiman trivial.
- **Measure yang additive disimpan, rasio dihitung saat query.** Simpan net_amount dan quantity; jangan simpan "AOV" di fact — rasio tidak bisa dijumlahkan lintas grup (jumlah AOV dua region bukan AOV gabungan).

■ TECHNICAL NOTE — Surrogate key & slowly changing dimension

Dimension memakai surrogate key (customer_key) terpisah dari natural key (customer_id) antara lain untuk menangani *slowly changing dimensions* (SCD): saat pelanggan naik segment dari Silver ke Gold, SCD Type 2 menyimpan dua baris berbeda periode validitas, sehingga penjualan lama tetap teratribusi ke "Silver saat itu". Tanpa ini, analisis historis per segment diam-diam ditulis ulang oleh keadaan sekarang. Cukup pahami konsepnya — implementasi mendalam adalah wilayah analytics engineer.

ETL vs ELT — bagaimana data berpindah

	ETL (klasik)	ELT (modern, dominan)
Urutan	Extract → Transform → Load	Extract → Load → Transform
Transformasi di	Server terpisah, sebelum masuk warehouse	Di dalam warehouse, dengan SQL
Kenapa menang	—	Warehouse kolumnar murah & kuat; transformasi = SQL yang analyst kuasai
Tool khas	Informatica, SSIS	Fivetran/Airbyte (EL) + dbt (T) + Airflow (orkestrasi)

Implikasi besarnya untuk kariermu: pada ELT, transformasi ditulis dalam SQL. Skill Bab 3-4 bukan hanya untuk analisis ad-hoc — ia adalah bahasa pembangunan warehouse itu sendiri. Analyst yang menguasai SQL + pipeline mindset dapat mengambil peran analytics engineer, salah satu jalur karier dengan permintaan tertinggi saat ini.

Pipeline mindset: dari query sekali pakai ke aset yang hidup

Mengapa ini pembeda kelas

Query ad-hoc menjawab satu pertanyaan satu kali. Pipeline menjawab pertanyaan yang sama setiap hari, otomatis, dengan kualitas terjaga. Perbedaan sikapnya sederhana tetapi mendalam — dan bisa kamu praktikkan bahkan sebelum menyentuh tool orkestrasi:

Kebiasaan ad-hoc	Pipeline mindset
Query di editor, hilang saat tutup	Query tersimpan di Git, ada riwayat perubahan
"Kemarin jalan kok"	Test otomatis: unique, not null, rentang nilai
Definisi metric di kepala	Definisi di kode + dokumentasi, satu tempat
Angka aneh → cek manual panik	Angka aneh → test gagal duluan, sebelum dashboard salah
Hardcode '2026-07-01'	Parameter periode; jalan untuk bulan apa pun

Empat test data minimum (konsep dbt, berlaku di tool apa pun)

```
-- 1. UNIQUENESS: PK benar-benar unik?
SELECT order_id, COUNT(*) FROM fct_orders
GROUP BY order_id HAVING COUNT(*) > 1;           -- harus 0 baris

-- 2. NOT NULL: kolom kritis tidak bolong?
SELECT COUNT(*) FROM fct_sales WHERE net_amount IS NULL;   -- harus 0

-- 3. ACCEPTED VALUES: kategori sesuai domain?
SELECT DISTINCT status FROM fct_orders
WHERE status NOT IN ('completed','cancelled','refunded','pending');

-- 4. RELATIONSHIP: semua FK punya induk? (orphan check)
SELECT f.order_id FROM fct_sales f
LEFT JOIN dim_product p ON p.product_key = f.product_key
WHERE p.product_key IS NULL;                        -- harus 0 baris
```

Empat query ini adalah penerapan langsung enam dimensi integrity Bab 2 — kini sebagai kode yang berjalan otomatis setiap kali data diperbarui. Di dbt, keempatnya cukup dideklarasikan dalam YAML; konsepnya persis sama.

✓ **BEST PRACTICE — Standar tim analytics matang**

✓ Semua transformasi di version control (Git) ✓ Naming convention konsisten: `fct_`, `dim_`, `mart_`, `stg_`

✓ Setiap model punya deskripsi + pemilik ✓ Test berjalan sebelum data dipublikasikan (fail loudly, fail early)

✓ Perubahan definisi metric melalui review, bukan edit diam-diam ✓ Dokumentasi hidup berdampingan dengan kode, bukan di wiki yang basi.

✗ **COMMON MISTAKE — Dashboard dibangun langsung di atas raw**

Gejala: dashboard penuh logika CASE rumit yang diduplikasi di lusinan chart; tiap perbaikan harus diulang di semua tempat; angka antar dashboard tak pernah cocok.

Mengapa terjadi: jalan pintas — membangun mart terasa lambat di awal.

Perbaikan: tarik logika bersama ke satu model core/mart; dashboard hanya memvisualkan.

Cara senior menghindarinya: aturan praktis — jika logika yang sama dibutuhkan dua tempat, ia harus naik satu lapisan.

Membaca kegagalan pipeline — skill debugging angka

Kembali ke kasus Bab 2: dashboard menunjukkan penjualan kemarin anjlok 40%. Protokol investigasi analyst yang paham arsitektur:

1. **Freshness:** kapan tabel terakhir ter-update? Bandingkan max(order_ts) dengan waktu sekarang.
2. **Volume:** jumlah baris kemarin vs rata-rata 7 hari — per sumber (POS vs e-commerce).
3. **Lineage:** jika satu sumber hilang, lapor ke pemilik pipeline dengan bukti; jangan umumkan "penjualan anjlok".
4. **Baru bicara bisnis:** jika data lengkap dan angka tetap turun, barulah itu sinyal bisnis — lanjut ke analisis.

```
-- Query freshness + volume harian per sumber, andalan investigasi:
SELECT
  source_system,
  DATE_TRUNC('day', order_ts) AS d,
  COUNT(*) AS row_cnt,
  SUM(net_amount) AS revenue
FROM fct_sales
WHERE order_ts >= CURRENT_DATE - INTERVAL '8 days'
GROUP BY 1, 2
ORDER BY 1, 2;
```

Checklist arsitektur & pipeline

- ✓ Analisis berjalan di warehouse/replica, bukan OLTP produksi
- ✓ Tabel yang dipakai berasal dari lapisan yang tepat (core/mart, bukan raw)
- ✓ Grain fact table dideklarasikan dan dipahami
- ✓ Rasio dihitung saat query, bukan disimpan sebagai measure
- ✓ Query yang dipakai berulang tersimpan di version control
- ✓ Empat test minimum (unique, not null, accepted values, relationship) terpasang pada dataset penting
- ✓ Investigasi angka aneh dimulai dari freshness & volume, bukan dari kesimpulan bisnis

Ending Card — Bab 5

What you learned

OLTP vs OLAP dan kolumnar vs row store; lapisan warehouse (raw → core → mart) dan single source of truth; star schema, fact/dimension, grain-first design, SCD; ETL vs ELT; pipeline mindset dengan empat test data minimum; protokol debugging angka anjlok.

Skills mastered

Membaca arsitektur data perusahaan · mendesain star schema sederhana · menulis data test SQL · debugging pipeline-first.

Real business value

Angka yang konsisten lintas laporan, insiden "dashboard salah" tertangkap sebelum dilihat eksekutif, dan jalur karier menuju analytics engineering terbuka.

Interview readiness

● INTERVIEW QUESTIONS

1. "Apa beda OLTP dan OLAP? Kenapa dipisah?" — sebutkan beban kerja yang saling membunuh.
2. "Desain star schema untuk bisnis retail" — mulai dari deklarasi grain fact; itu yang dinilai.
3. "Kenapa industri bergeser ETL → ELT?" — warehouse kolumnar murah + transformasi SQL.
4. "Dashboard menunjukkan penurunan drastis mendadak. Langkah pertamamu?" — jawaban kuat: cek freshness/volume pipeline dulu, bukan langsung teori bisnis.

Exercises

Exercise 5.1 (Intermediate · 30 menit)

Objective: desain star schema

Desain star schema untuk bisnis F&B (kedai kopi 20 cabang): tentukan grain fact table penjualan, kolom fact, dan minimal 4 dimension beserta kolom pentingnya. Jelaskan satu keputusan grain yang sulit (petunjuk: satu transaksi berisi beberapa menu; modifier seperti "extra shot" mau ditaruh di mana?).

Answer key 5.1 (jawaban kuat)

Grain: 1 baris = 1 line item per struk (menu per transaksi). Fact: date_key, time_key, store_key, product_key, cashier_key, quantity, gross_amount, discount_amount, net_amount. Dimensions: dim_date (+is_weekend/holiday), dim_store (lokasi, tipe), dim_product (menu, kategori, ukuran), dim_time (jam — penting untuk peak hour F&B). Keputusan modifier: opsi kuat (a) baris terpisah ber-flag is_modifier dengan parent_item_id, atau (b) atribut di dim_product sebagai varian. (a) lebih fleksibel untuk analisis attachment rate modifier; (b) lebih sederhana. Yang dinilai: kesadaran trade-off, bukan jawaban tunggal.

Exercise 5.2 (Advanced · 30 menit)

Objective: menulis data tests

Untuk fct_sales TokoNusa, tulis 5 data test SQL: PK unik, net_amount tidak negatif kecuali refund, tidak ada orphan product_key, discount tidak melebihi gross, dan freshness (data hari kemarin ada). Tentukan juga: test mana yang harus menghentikan publikasi data (blocking) vs hanya warning?

Answer key 5.2 (inti + klasifikasi)

Kelima test mengikuti pola bab ini; contoh discount: `SELECT COUNT(*) FROM fct_sales WHERE discount_amount > gross_amount` — harus 0. Freshness: `SELECT COUNT(*) FROM fct_sales WHERE order_ts >= CURRENT_DATE - INTERVAL '1 day'` — harus > 0. Klasifikasi yang dipertahankan: PK unik & orphan = blocking (merusak semua agregasi); discount > gross = blocking (data korup); net negatif = warning + investigasi (bisa sah untuk refund); freshness = warning pagi, blocking menjelang jam laporan. Yang dinilai: kesadaran bahwa severity adalah keputusan bisnis, bukan teknis semata.

Portfolio idea

★ PORTFOLIO TIP

Proyek kecil ber-impact besar: ambil dataset publik transaksional, modelkan star schema-nya dengan dbt (gratis, dokumentasinya sangat baik), lengkapi 4 test minimum, dan tulis README berisi diagram skema. "Junior yang mengerti dbt + testing" adalah profil yang dicari-cari tim data saat ini.

Further reading

Kimball & Ross, *The Data Warehouse Toolkit* (bab fact/dimension) · dbt docs: "Best practices" · *Fundamentals of Data Engineering* (Reis & Housley) untuk gambaran besar.

Next chapter

Bab 6 turun ke pekerjaan paling memakan waktu dalam profesi ini: data preparation — membersihkan kekacauan dunia nyata menjadi dataset analitis, dengan framework yang sistematis.

Bab 6 — Data Preparation

Framework sistematis mengubah kekacauan dunia nyata menjadi dataset analitis

Learning objectives	Menjalankan profiling sebelum cleaning; menangani missing values, duplicate, kategori tidak konsisten, dan outlier dengan keputusan yang terdokumentasi; konversi tipe dan transformasi; membangun feature analitis (RFM, tenure, flag perilaku)
Prerequisites	Bab 2-4; dasar Python membantu (dilengkapi Bab 7)
Estimasi waktu	Baca 2,5 jam · Praktik 6 jam
Difficulty	● Intermediate
Skills covered	Data profiling, cleaning, missing values, dedup, outlier detection, type conversion, feature creation
Business domain	Retail (TokoNusa)

Mengapa preparation adalah 60-80% pekerjaan — dan mengapa itu wajar

Survei industri konsisten selama satu dekade: analyst menghabiskan mayoritas waktunya menyiapkan data, bukan menganalisisnya. Ini bukan inefisiensi yang harus dihilangkan — ini *tempat kualitas analisis ditentukan*. Model statistik terbaik atas data kotor menghasilkan kesimpulan kotor dengan kepercayaan diri tinggi.

Yang membedakan senior bukan menghindari pekerjaan ini, melainkan mengerjakannya sistematis: framework tetap, keputusan terdokumentasi, hasil yang bisa direproduksi.

Framework preparation lima langkah

1. PROFILE kenali data sebelum menyentuhnya
▼ (shape, null rate, distribusi, kardinalitas)
 2. VALIDATE bandingkan dengan aturan bisnis
▼ (rentang sah, domain nilai, relasi antar kolom)
 3. CLEAN tangani missing, duplicate, inkonsistensi, outlier
▼ (setiap keputusan DICATAT)
 4. TRANSFORM tipe data, standardisasi, struktur analitis
▼
 5. ENRICH feature creation: RFM, tenure, flag, rasio
▼
- Analytical dataset + dokumentasi + log keputusan

✓ **BEST PRACTICE — Aturan emas preparation**

Jangan pernah menimpa data mentah. Alur selalu: raw (read-only) → script transformasi → dataset bersih. Jika cleaning-mu tidak bisa diulang dari nol dengan satu perintah, ia belum selesai. Ini penerapan langsung pipeline mindset Bab 5.

Langkah 1 — Profiling: kenali sebelum menyentuh

Python (pandas) – profil cepat standar; pola SQL setara diberikan setelahnya

```
import pandas as pd

df = pd.read_csv("raw_customers.csv")

df.shape           # (2_143_055, 9) – baris, kolom
df.dtypes          # tipe tiap kolom – sering salah tebak!
df.isna().mean().round(3) # % null per kolom
df.describe()      # statistik kolom numerik: cek min/max mustahil
df["city"].value_counts().head(20) # kardinalitas & variasi ejaan
df.duplicated().mean() # % exact duplicate
```

Enam baris ini menjawab: seberapa besar, seberapa bolong, seberapa aneh. Temuan khas di data nyata: umur 208 tahun, harga negatif, "Jakarta" dalam lima ejaan, tanggal tahun 1970 (default Unix epoch — tanda nilai kosong yang terisi paksa).

Langkah 3a — Missing values: putuskan, jangan refleksi

Diagnosis dulu: mengapa hilang?

Perlakuan yang benar bergantung pada mekanisme hilangnya — kembali ke tabel makna NULL di Bab 2. Pertanyaan kuncinya: **apakah hilangnya acak atau berpola?** Cek dengan membandingkan kelompok:

```
# Apakah baris tanpa city berbeda perilaku dari yang lengkap?
df.groupby(df["city"].isna())["total_spend"].describe()
# Jika spend kelompok NULL jauh berbeda → hilangnya BERPOLA;
# membuang barisnya akan membiaskan analisis
```

Empat opsi penanganan dan kapan dipakai

Opsi	Kapan tepat	Risiko
Biarkan NULL + flag	Default terbaik untuk analisis deskriptif	Harus disiplin di setiap agregasi
Buang baris	Hilang sedikit ($< \pm 5\%$) DAN terbukti acak	Bias jika berpola; catat berapa yang dibuang
Isi konstanta ("Unknown", 0)	Categorical: kategori "Unknown" eksplisit	Mengisi 0 pada numerik merusak mean — hampir selalu salah
Imputasi (median/ model)	Kebutuhan ML (Semester 6)	Menyembunyikan ketidakpastian; jarang tepat untuk BI

✗ COMMON MISTAKE — fillna(0) refleksi

Gejala: rata-rata belanja pelanggan anjlok misterius setelah cleaning.

Mengapa terjadi: NULL "belum pernah belanja bulan ini" diisi 0, lalu dirata-rata bersama pembeli aktif — mencampur dua populasi berbeda.

Perbaikan: pisahkan pertanyaan: "rata-rata belanja *pembeli aktif*" (NULL dikeluarkan) vs "rata-rata belanja *semua pelanggan*" (0 sah). Dua metric, dua makna — laporkan yang sesuai pertanyaan.

Cara senior menghindarinya: sebelum mengisi apa pun, tulis satu kalimat: "NULL di kolom ini berarti ___, maka perlakuannya ___".

Langkah 3b — Duplicate: exact dan logical

```
# Exact duplicate: aman dibuang, tapi CATAT jumlahnya
n_before = len(df)
df = df.drop_duplicates()
print(f"Dropped {n_before - len(df)} exact duplicates")

# Logical duplicate: entitas sama, baris beda — perlu normalisasi dulu
df["email_norm"] = df["email"].str.strip().str.lower()
dupes = df[df.duplicated(subset="email_norm", keep=False)]
dupes.sort_values("email_norm").head(10)      # inspeksi manual sampel dulu!

# Aturan resolusi harus DIPUTUSKAN, contoh: simpan record ter-update
df = (df.sort_values("updated_at", ascending=False)
      .drop_duplicates(subset="email_norm", keep="first"))
```

Perhatikan urutannya: normalisasi → inspeksi sampel → aturan resolusi eksplisit → eksekusi. Versi SQL dari langkah terakhir adalah pola ROW_NUMBER Bab 4 — teknik yang sama, dua bahasa.

◆ BUSINESS INSIGHT

Keputusan "record mana yang disimpan" adalah keputusan bisnis, bukan teknis. Menyimpan alamat terbaru benar untuk pengiriman; menyimpan record pertama benar untuk atribusi akuisisi. Tanyakan pemilik proses sebelum memutuskan — lalu dokumentasikan.

Langkah 3c — Kategori tidak konsisten

"Jakarta", "JAKARTA", "Jakarta ", "DKI Jakarta", "Jkt" — lima ejaan, satu kota, lima grup berbeda di GROUP BY. Protokol standardisasi:

```
# 1. Lihat semua variasi + frekuensinya
df["city"].value_counts()

# 2. Normalisasi mekanis dulu (murah, aman)
df["city"] = df["city"].str.strip().str.title()

# 3. Mapping eksplisit untuk sisanya — mapping = DOKUMENTASI
city_map = {"Dki Jakarta": "Jakarta", "Jkt": "Jakarta",
            "Bdg": "Bandung", "Sby": "Surabaya"}
df["city"] = df["city"].replace(city_map)

# 4. Verifikasi: kardinalitas turun ke jumlah yang masuk akal?
df["city"].nunique()
```

Simpan `city_map` di file terpisah yang di-version control. Bulan depan data baru datang dengan variasi baru — mapping yang terdokumentasi tinggal ditambah, bukan dibuat ulang dari nol.

Langkah 3d — Outlier: deteksi, lalu KLASIFIKASI, baru putuskan

Deteksi standar

```
# Metode IQR (robust, default yang baik untuk data bisnis yang skewed)
q1, q3 = df["order_value"].quantile([0.25, 0.75])
iqr = q3 - q1
upper = q3 + 1.5 * iqr
```



```
outliers = df[df["order_value"] > upper]
print(f"{len(outliers)} outliers ({len(outliers)/len(df):.2%})")
```

■ TECHNICAL NOTE

Metode z-score ($|z| > 3$) mengasumsikan distribusi normal — buruk untuk data bisnis yang hampir selalu right-skewed (sedikit transaksi sangat besar). IQR dan percentile-based lebih aman. Dasar statistiknya diformalkan di Semester 2.

Klasifikasi: tiga jenis outlier, tiga perlakuan berbeda

Jenis	Contoh	Perlakuan
Error data	quantity = 5000 (typo kasir), harga negatif non-refund	Perbaiki dari sumber atau buang + catat
Kejadian sah ekstrem	Order korporat 500 juta; lonjakan Harbolnas	JANGAN dibuang — sering justru paling penting bisnisnya
Populasi berbeda	Pembeli B2B tercampur di data retail	Pisahkan sebagai segmen; analisis terpisah

⚠ WARNING

Membuang outlier tanpa investigasi adalah cara elegan menghapus sinyal bisnis paling berharga. Pembeli terbesar, kegagalan sistem termahal, dan awal tren baru semuanya lahir sebagai "outlier". Aturan: outlier boleh dikeluarkan dari sebuah metric hanya dengan alasan tertulis, dan keberadaannya tetap dilaporkan.

Langkah 4 — Type conversion & transformasi

```
# Tanggal: string → datetime, dengan errors="coerce" agar yang gagal jadi NaT
df["join_date"] = pd.to_datetime(df["join_date"], errors="coerce")
df["join_date"].isna().sum()      # berapa yang gagal parse? selidiki polanya

# Angka yang tersimpan sebagai teks: "Rp1.250.000" → 1250000
df["price"] = (df["price"].str.replace(r"[Rp.\s]", "", regex=True)
               .astype("float64"))

# Identifier: paksa jadi string — pelajaran Bab 2
df["store_id"] = df["store_id"].astype("string")
```

`errors="coerce"` lebih baik daripada membiarkan satu nilai rusak menggagalkan seluruh konversi — tetapi wajib diikuti penghitungan berapa yang menjadi NULL dan mengapa. Konversi yang menelan 3% data secara diam adalah bug, bukan solusi.

Langkah 5 — Feature creation: dari kolom mentah ke sinyal analitis

Mengapa

Nilai analisis sering bukan di kolom asli, tetapi di turunannya: tanggal lahir → umur; tanggal join → tenure; riwayat transaksi → RFM. Feature yang baik menerjemahkan perilaku menjadi angka yang bisa dianalisis.

Production example — membangun tabel RFM (fondasi segmentasi retail)

RFM = Recency (kapan terakhir), Frequency (seberapa sering), Monetary (berapa besar)

```

snapshot = pd.Timestamp("2026-08-01")

rfm = (orders[orders["status"] == "completed"]
      .groupby("customer_id")
      .agg(last_order = ("order_ts", "max"),
           frequency = ("order_id", "nunique"),
           monetary = ("net_amount", "sum"))
      .reset_index())

rfm["recency_days"] = (snapshot - rfm["last_order"]).dt.days

# Skor 1-5 per dimensi dengan quintile; 5 = terbaik
rfm["R"] = pd.qcut(rfm["recency_days"], 5, labels=[5,4,3,2,1]).astype(int)
rfm["F"] = pd.qcut(rfm["frequency"].rank(method="first"), 5, labels=[1,2,3,4,5]).astype(int)
rfm["M"] = pd.qcut(rfm["monetary"], 5, labels=[1,2,3,4,5]).astype(int)

# Segmen bisnis dari kombinasi skor
def segment(row):
    if row.R >= 4 and row.F >= 4: return "Champions"
    if row.R <= 2 and row.F >= 4: return "At Risk (dulu loyal)"
    if row.R >= 4 and row.F <= 2: return "New / Promising"
    if row.R <= 2 and row.F <= 2: return "Hibernating"
    return "Regular"

rfm["segment"] = rfm.apply(segment, axis=1)
rfm["segment"].value_counts(normalize=True).round(3)

```

Expected output (proporsi khas retail sehat)

```

Regular          0.36
Hibernating      0.24
Champions        0.15
New / Promising  0.14
At Risk (dulu loyal) 0.11

```

◆ BUSINESS INSIGHT — RFM langsung menjadi tindakan

Champions → program loyalti & early access. At Risk → winback sebelum benar-benar hilang (jauh lebih murah daripada akuisisi baru). New/Promising → dorong pembelian kedua — transisi 1→2 pembelian biasanya lompatan retensi terbesar. Inilah contoh dataset yang "tahu apa yang harus dilakukan" — hasil preparation yang baik hampir menganalisis dirinya sendiri.

Feature analitis lain yang rutin dibuat

Feature	Dari	Untuk
tenure_months	join_date	Analisis kohort & loyalitas
days_since_last_order	max(order_ts)	Deteksi churn dini
aov per customer	monetary / frequency	Profil nilai pelanggan
preferred_channel / category	mode transaksi	Personalisasi
is_weekend_shopper	EXTRACT(DOW)	Penjadwalan kampanye
discount_dependency	% order berdiskon	Risiko margin per segmen

Dokumentasi: cleaning log dan data dictionary

Dua artefak wajib menyertai setiap analytical dataset:

```
CLEANING LOG – analytical_customers v1.2 (2026-08-01)
```

1. 8.211 exact duplicates dibuang (0,4%)
2. 31.874 logical duplicates by email_norm → keep latest updated_at
3. city: 41 variasi → 12 nilai standar (mapping: city_map.csv)
4. 2,1% baris tanpa join_date → dipertahankan, flag join_date_missing
5. 214 order_value > P99.9 diverifikasi: 197 order korporat sah (flag is_b2b), 17 error input → dibuang, dilaporkan ke ops

Data dictionary minimal berisi: nama kolom, tipe, definisi bisnis satu kalimat, sumber, dan aturan NULL. Format lengkapnya dipraktikkan di capstone Bab 8.

Checklist data preparation

- ✓ Data mentah tidak pernah ditimpa; seluruh cleaning dalam script yang bisa diulang
- ✓ Profiling dilakukan sebelum cleaning, temuan dicatat
- ✓ Mekanisme missing (acak vs berpola) dicek sebelum memilih perlakuan
- ✓ Dedup: normalisasi → inspeksi sampel → aturan resolusi eksplisit
- ✓ Mapping kategori tersimpan sebagai file ter-version-control
- ✓ Outlier diklasifikasi (error / sah / populasi lain) sebelum diputuskan
- ✓ Konversi tipe menghitung dan menyelidiki nilai yang gagal
- ✓ Cleaning log + data dictionary menyertai dataset final

Ending Card — Bab 6

What you learned

Framework lima langkah (profile → validate → clean → transform → enrich); diagnosis mekanisme missing; dedup dua tingkat; standardisasi kategori dengan mapping terdokumentasi; klasifikasi outlier tiga jenis; type conversion defensif; feature creation termasuk RFM lengkap; cleaning log.

Skills mastered

Profiling cepat · keputusan cleaning yang defensibel · pembangunan feature analitis · dokumentasi preparation.

Real business value

Dataset yang dipercaya adalah prasyarat semua analisis hilir. RFM saja sudah cukup untuk menggerakkan program CRM bernilai miliaran — langsung dari materi bab ini.

Interview readiness

● INTERVIEW QUESTIONS

1. "Bagaimana kamu menangani missing values?" — jawaban kuat SELALU dimulai dari "tergantung mekanismenya" + cara mengeceknya. Jawaban lemah: langsung menyebut imputation.
2. "Kapan outlier boleh dibuang?" — hanya error terverifikasi; kejadian sah ekstrem dipertahankan.
3. "Ceritakan dataset terkotor yang pernah kamu tangani" — struktur jawaban: profiling → temuan → keputusan → dampak.
4. "Apa itu RFM dan untuk apa?" — definisi + 2 contoh aksi bisnis per segmen.

Exercises

Exercise 6.1 (Beginner · 20 menit)

Objective: keputusan missing values

Kolom phone_number kosong 34%. Tim CRM ingin kampanye SMS. Apa yang kamu periksa, dan apa rekomendasi realistikmu?

Answer key 6.1

Periksa: (a) pola — apakah kosong terkonsentrasi di periode/channel pendaftaran tertentu (form lama tanpa field wajib)? (b) validitas nomor yang ada (format, panjang). Rekomendasi: kampanye hanya menjangkau $\pm 66\%$ basis — laporkan coverage ini eksplisit; jangan "membersihkan" dengan membuang baris kosong (itu menghapus sepertiga pelanggan dari SEMUA analisis lain). Usulkan perbaikan hulu: field wajib + validasi di form. Poin kunci: solusi missing data terbaik sering di proses bisnis, bukan di pandas.

Exercise 6.2 (Intermediate · 40 menit)

Objective: investigasi outlier

Distribusi order_value TokoNusa: median 185 ribu, P99 2,1 juta, maksimum 890 juta. Ada 214 order > 50 juta. Susun protokol investigasi lengkap sebelum memutuskan perlakuan, dan tentukan dampaknya ke metric AOV yang dilaporkan bulanan.

Answer key 6.2

Protokol: (1) inspeksi baris — siapa pembelinya, produk apa, kasir/channel mana; (2) cek pola: pembeli berulang dengan NPWP → indikasi B2B; satu kasir yang sama berulang → indikasi error input/fraud; (3) konfirmasi ke tim toko untuk sampel. Keputusan khas: mayoritas B2B sah → beri flag `is_b2b`, JANGAN dibuang. Dampak metric: laporkan AOV retail (exclude B2B) dan revenue total (include) secara terpisah — mencampur keduanya membuat AOV melonjak-longsor mengikuti order korporat dan tak lagi menggambarkan perilaku konsumen. Lampirkan definisi kedua metric di laporan.

Exercise 6.3 — Production challenge (Advanced · 90 menit)

Objective: preparation end-to-end

Ambil dataset transaksi publik yang kotor (mis. "Online Retail II" di UCI — berisi retur, harga negatif, customer NULL). Jalankan framework lima langkah penuh, hasilkan: script yang reproducible, dataset bersih, tabel RFM, cleaning log, dan data dictionary.

Answer key 6.3 (kriteria penilaian)

Kualitas dinilai dari keputusan yang terdokumentasi, bukan kebersihan sempurna: retur (quantity negatif) dipisahkan bukan dibuang — mereka data bisnis penting; transaksi tanpa `customer_id` dikeluarkan dari RFM tetapi tetap dihitung di revenue (dua populasi, dua kegunaan); harga 0 diselidiki (sampel gratis? error?). Kesalahan umum: drop semua baris "aneh" di awal — menghapus 20%+ data tanpa memahami artinya, membuat revenue tidak lagi rekonsiliasi dengan sumber.

Portfolio idea

★ PORTFOLIO TIP

Exercise 6.3 adalah artefak portofolio kelas atas jika dikemas benar: notebook dengan narasi keputusan (bukan hanya kode), cleaning log, dan README "before/after". Recruiter teknis membaca cara kamu MEMUTUSKAN, bukan sintaksmu.

Further reading

pandas User Guide: "Working with missing data" · *Bad Data Handbook* (Q. E. McCallum) · Artikel klasik Hadley Wickham, "Tidy Data" (Journal of Statistical Software).

Next chapter

Bab 7 melengkapi toolkit: kapan Excel, kapan Python, kapan SQL — dan workflow pandas yang menjadi standar kerja harian analyst.

Bab 7 — Excel & Python untuk Analyst

Kapan pakai apa — dan workflow pandas yang menjadi standar kerja harian

Learning objectives	Memilih tool yang tepat per situasi (SQL vs Excel vs Python); menguasai Excel level analyst (tables, PivotTable, XLOOKUP, Power Query); menguasai workflow inti pandas; menghindari jebakan klasik kedua tool
Prerequisites	Bab 3-6
Estimasi waktu	Baca 2 jam · Praktik 6 jam
Difficulty	● Beginner → ● Intermediate
Skills covered	Excel tables, PivotTable, XLOOKUP, Power Query, pandas core, groupby, merge, workflow reproducible
Business domain	Retail (TokoNusa)

Mengapa tiga tool, bukan satu

Pertanyaan "Excel atau Python?" adalah pertanyaan yang salah. Analyst profesional memakai ketiganya setiap minggu, untuk pekerjaan yang berbeda. Yang membedakan senior adalah *refleks memilih* — dan disiplin untuk tidak memaksakan tool favorit ke masalah yang salah.

Situasi	Tool tepat	Alasan
Data > ~500 ribu baris, sumber = warehouse	SQL	Komputasi terjadi di server; jangan tarik jutaan baris ke laptop
Agregasi/JOIN atas data warehouse	SQL	Bahasa asli warehouse; hasil bisa dipakai ulang tim
Eksplorasi cepat file kecil; angka final untuk dicek finance	Excel	Kecepatan inspeksi visual tak tertandingi; bahasa bersama seluruh kantor
Cleaning kompleks, logika bercabang, harus diulang tiap bulan	Python	Reproducible, testable, version-controlled
Statistik, feature engineering, otomasi laporan	Python	Ekosistem pandas/scipy; jembatan ke Semester 2 & 6
Deliverable untuk stakeholder non-teknis yang ingin mengutak-atik	Excel	Serahkan dalam format yang bisa MEREKA pakai

► EXECUTIVE PERSPECTIVE

Finance dan operasional hidup di Excel. Analyst yang menyerahkan file .ipynb kepada CFO tidak akan dibaca; analyst yang menyerahkan Excel rapi dengan angka yang bisa ditelusuri akan dipercaya. Tool terbaik adalah yang membuat keputusan terjadi.

Excel level analyst — lima kemampuan yang benar-benar dipakai

1. Excel Tables (Ctrl+T) — fondasi yang dilewatkan pemula

Mengubah range menjadi Table memberi: nama kolom stabil, formula yang menyalin otomatis, dan referensi terstruktur (`=SUM(Sales[net_amount])`) yang tidak rusak saat data bertambah. Semua fitur lain (PivotTable, Power Query) bekerja lebih baik di atas Table. Kebiasaan pertama: data masuk → jadikan Table → beri nama.

2. PivotTable — GROUP BY-nya Excel

PivotTable adalah "measure by dimension" (Bab 2) dalam bentuk drag-and-drop: dimension ke Rows/Columns, measure ke Values. Kemampuan yang perlu dikuasai: value field settings (sum vs count vs % of total), grouping tanggal per bulan, dan slicer untuk filter interaktif. Latihan terbaik: replikasi hasil query GROUP BY Bab 3 di PivotTable — sampai kamu melihat keduanya sebagai operasi yang sama.

3. XLOOKUP — JOIN-nya Excel

```
=XLOOKUP(A2; Products[product_id]; Products[category]; "Not found")  
// cari A2 di kolom product_id, kembalikan category; default jika tak ketemu
```

XLOOKUP menggantikan VLOOKUP lama: bisa mencari ke kiri, default value eksplisit, tidak rusak saat kolom disisipkan. Secara konsep ini LEFT JOIN satu kolom — dan jebakan NULL-nya pun sama: selalu isi argumen "if_not_found", jangan biarkan #N/A menyebar ke perhitungan.

4. Formula agregasi bersyarat

```
=SUMIFS(Sales[net]; Sales[region]; "Jakarta"; Sales[month]; 7)  
=COUNTIFS(Cust[segment]; "Gold"; Cust[city]; "Bandung")  
// padanan langsung conditional aggregation Bab 3
```

5. Power Query — pipeline di dalam Excel

Power Query (Data → Get & Transform) adalah alat paling underrated di Excel: impor dari file/database/folder, langkah cleaning yang *terekam dan bisa diulang* (rename, filter, unpivot, merge), lalu refresh sekali klik saat data baru datang. Ini pipeline mindset Bab 5 versi Excel — dan jembatan alami ke Power BI di Semester 2. Jika kamu masih membersihkan data dengan copy-paste manual tiap bulan, Power Query adalah upgrade terbesarmu.

✗ COMMON MISTAKE — Excel sebagai database

Gejala: file "MASTER_DATA_FINAL_v7 (recovered).xlsx" 180 MB, 40 sheet saling refer, rumus tak ada yang berani sentuh, angka berbeda tergantung siapa yang buka.

Mengapa terjadi: solusi cepat yang tumbuh 3 tahun tanpa arsitektur.

Perbaikan: data historis → database/warehouse; Excel untuk analisis dan penyajian, bukan penyimpanan.

Cara senior menghindarinya: aturan praktis — jika file dibuka lebih dari satu orang untuk di-*edit*, atau riwayatnya penting, ia bukan lagi pekerjaan Excel.

Python untuk analyst — workflow pandas inti

Mengapa pandas

pandas adalah "Excel yang bisa diprogram": tabel (DataFrame) dengan operasi filter, pivot, join, dan agregasi — tetapi reproducible, cepat di ratusan ribu-jutaan baris, dan tersambung ke seluruh ekosistem Python (statistik Semester 2, ML Semester 6).

Peta konsep: SQL ↔ pandas ↔ Excel

Operasi	SQL	pandas	Excel
Filter baris	WHERE	<code>df[df.col > x]</code>	Filter / SUMIFS
Pilih kolom	SELECT a, b	<code>df[["a","b"]]</code>	—
Agregasi per grup	GROUP BY	<code>groupby().agg()</code>	PivotTable
Gabung tabel	JOIN	<code>merge()</code>	XLOOKUP
Urutkan	ORDER BY	<code>sort_values()</code>	Sort
Kolom baru	SELECT expr AS	<code>df.assign() / df["x"]=</code>	Formula kolom
Window/ranking	OVER(PARTITION BY)	<code>groupby().transform/rank</code>	—

Satu peta ini mempercepat belajar berkali lipat: kamu tidak sedang belajar tiga hal, melainkan satu hal dalam tiga bahasa.

Business example — laporan mingguan dalam satu script

Business question (weekly ops review): revenue & order per region per minggu, dibandingkan minggu sebelumnya

```
import pandas as pd

# 1. LOAD — dari hasil query SQL atau file ekspor
orders = pd.read_parquet("fct_orders.parquet")
stores = pd.read_csv("dim_store.csv")

# 2. PREP — filter status & periode (disiplin Bab 3 berlaku di sini juga)
completed = orders.query("status == 'completed'").copy()
completed["week"] = completed["order_ts"].dt.to_period("W").dt.start_time

# 3. JOIN — selalu cek fan-out dengan validate!
df = completed.merge(stores[["store_id", "region"]],
                    on="store_id", how="left", validate="many_to_one")

# 4. AGGREGATE — grain hasil: region × week
weekly = (df.groupby(["region", "week"], as_index=False)
          .agg(revenue=("net_amount", "sum"),
              orders=("order_id", "nunique")))

# 5. COMPARE — LAG versi pandas: shift per grup
weekly = weekly.sort_values(["region", "week"])
weekly["rev_prev"] = weekly.groupby("region")["revenue"].shift(1)
weekly["wow_growth"] = ((weekly["revenue"] / weekly["rev_prev"]) - 1).round(3)

# 6. VALIDATE — rekonsiliasi sebelum kirim (protokol Bab 4)
assert abs(weekly["revenue"].sum() - completed["net_amount"].sum()) < 1
```



```
# 7. DELIVER — dalam format audiens: Excel untuk ops team
weekly.to_excel("weekly_region_report.xlsx", index=False)
```

Tujuh langkah ini — load, prep, join, aggregate, compare, validate, deliver — adalah kerangka hampir semua script analisis. Perhatikan dua kebiasaan produksi: `validate="many_to_one"` membuat merge *gagal keras* jika `dim_store` ternyata mengandung duplikat (fan-out Bab 2 tertangkap otomatis), dan `assert` rekonsiliasi menjaga total tetap jujur.

✗ COMMON MISTAKE — merge yang menggandakan baris diam-diam

Gejala: revenue naik misterius setelah merge; baris bertambah dari 1,2 juta menjadi 1,9 juta.

Mengapa terjadi: tabel lookup mengandung duplikat key — setiap duplikat menggandakan baris kiri; pandas tidak memperingatkan apa pun secara default.

Perbaiki: selalu pakai `validate=` ("one_to_one", "many_to_one") dan bandingkan `len(df)` sebelum-sesudah.

Cara senior menghindarinya: refleksi tiga baris setiap merge: shape sebelum, shape sesudah, assert.

■ TECHNICAL NOTE — SettingWithCopyWarning

Peringatan pandas paling terkenal ini muncul saat menulis ke "potongan" DataFrame yang statusnya ambigu (view atau copy). Cara aman yang menghilangkan seluruh kelas masalah ini: setiap membuat subset yang akan dimodifikasi, tambahkan `.copy()` — seperti pada `completed = orders.query(...).copy()` di atas. Untuk memodifikasi sebagian baris: `df.loc[kondisi, "col"] = nilai`, satu operasi, bukan dua kali indexing.

▲ PERFORMANCE TIP

(1) Baca hanya kolom yang dipakai: `pd.read_parquet(f, columns=[...])`. (2) Parquet, bukan CSV, untuk data antar-tahap — lebih kecil, lebih cepat, tipe data terjaga. (3) Hindari `df.apply(..., axis=1)` pada jutaan baris — operasi vectorized (kolom langsung) puluhan-ratusan kali lebih cepat. (4) Data terlalu besar untuk memori? Kembalikan agregasi ke SQL — itu tugasnya.

Reproducibility: struktur kerja minimal

```
project_payday_analysis/
├── README.md           ← pertanyaan bisnis, cara menjalankan, temuan utama
├── data/
│   ├── raw/           ← read-only, tidak pernah diedit
│   └── processed/      ← hasil script, boleh dihapus & dibuat ulang
├── sql/
│   └── extract_orders.sql
├── notebooks/
│   └── 01_explore.ipynb ← eksplorasi; TIDAK jadi sumber angka final
├── src/
│   └── build_report.py ← pipeline final: raw → laporan, satu perintah
└── output/
    └── weekly_region_report.xlsx
```

Prinsipnya satu: **notebook untuk berpikir, script untuk menghasilkan**. Notebook penuh sel yang dijalankan acak adalah sumber "angka yang tidak bisa direproduksi minggu depan". Angka yang dikirim ke stakeholder harus keluar dari script yang berjalan atas-ke-bawah tanpa intervensi.

Checklist Excel & Python

- ✓ Pilihan tool diputuskan dari ukuran data, kebutuhan pengulangan, dan audiens

- ✓ Excel: data sebagai Table; lookup dengan XLOOKUP + if_not_found; cleaning berulang lewat Power Query
- ✓ pandas: setiap merge memakai validate= dan cek shape sebelum-sesudah
- ✓ Subset yang dimodifikasi dibuat dengan .copy(); penulisan memakai .loc
- ✓ Angka final keluar dari script yang reproducible, bukan notebook acak
- ✓ Rekonsiliasi total (assert) sebelum deliverable dikirim
- ✓ Deliverable dalam format yang audiens bisa pakai

Ending Card — Bab 7

What you learned

Matriks pemilihan tool; lima kemampuan Excel level analyst; peta konsep SQL↔pandas↔Excel; workflow tujuh langkah pandas dengan kebiasaan validasi; struktur proyek reproducible.

Skills mastered

Tool selection · Excel analitis (Table, Pivot, XLOOKUP, Power Query) · pandas produksi (merge tervalidasi, groupby, shift) · disiplin reproducibility.

Real business value

Laporan berulang yang tadinya sehari kerja manual menjadi satu perintah — dan bebas dari kesalahan copy-paste yang menghancurkan kredibilitas.

Interview readiness

● INTERVIEW QUESTIONS

1. "Kapan kamu pakai SQL vs pandas?" — jawab dengan lokasi komputasi (server vs laptop) + reproducibility.
2. "Apa yang bisa salah saat merge dua tabel?" — duplikat key → fan-out; sebut validate= dan cek shape.
3. "Bagaimana memastikan analisis Python-mu bisa diulang orang lain?" — struktur proyek, script vs notebook, data raw read-only.
4. Live test umum: diberi CSV kotor, diminta agregasi per grup dengan pembanding periode — persis workflow tujuh langkah bab ini.

Exercises

Exercise 7.1 (Beginner · 20 menit)

Objective: refleksi pemilihan tool

Untuk tiap skenario, pilih tool utama + alasan satu kalimat: (a) CFO minta cek cepat 3 angka dari file 2.000 baris, meeting 30 menit lagi; (b) laporan bulanan dari warehouse 40 juta baris/bulan, diulang tiap bulan; (c) cleaning 12 file Excel format seragam dari 12 cabang, tiap minggu; (d) one-off: hitung korelasi diskon vs quantity pada 300 ribu baris hasil ekspor.

Answer key 7.1

(a) Excel — kecepatan inspeksi, taruhan rendah. (b) SQL untuk ekstraksi + agregasi di warehouse; Python bila perlu orkestrasi/format — jangan tarik 40 juta baris ke laptop. (c) Power Query (impor folder, langkah terekam, refresh sekali klik) atau Python bila logika bercabang. (d) Python/pandas — 300 ribu baris nyaman di memori dan butuh fungsi statistik.

Exercise 7.2 (Intermediate · 45 menit)

Objective: workflow pandas penuh

Dengan data TokoNusa (atau dataset publik setara): bangun script yang menghasilkan laporan "top 5 kategori per region bulan lalu + growth vs 2 bulan lalu", ekspor ke Excel, dengan minimal 2 assert validasi. Sebutkan di komentar: grain tiap tahap.

Answer key 7.2 (kriteria)

Struktur kuat: load → filter periode → merge tervalidasi → groupby(region, category, month) → shift(1) per (region, category) untuk growth → rank per (region, month) → filter rank ≤ 5 → ekspor. Assert yang diharapkan: rekonsiliasi total revenue vs data sumber; tidak ada null region setelah merge. Poin ekstra: parameter bulan (bukan hardcoded) — pipeline mindset Bab 5.

Portfolio idea

★ PORTFOLIO TIP

Repositori "automated weekly report": README bisnis, struktur folder standar bab ini, satu perintah menghasilkan Excel final dari raw. Sertakan screenshot output. Ini menjawab pertanyaan interview "pernah mengotomasi sesuatu?" dengan bukti, bukan cerita.

Further reading

pandas User Guide: "10 minutes to pandas" lalu "Merge, join, concatenate" · *Python for Data Analysis* (Wes McKinney)
· Dokumentasi Power Query M (Microsoft Learn).

Next chapter

Bab 8 — capstone: seluruh skill Semester 1 dirangkai menjadi satu proyek utuh, dari data mentah sampai analytical dataset terdokumentasi plus komunikasi hasil ke stakeholder.

Bab 8 — Capstone Semester 1

Dari data mentah ke analytical dataset terdokumentasi — plus komunikasi yang membuat kerja terlihat

Learning objectives	Merangkai seluruh skill semester menjadi proyek utuh; menghasilkan analytical dataset + data dictionary + cleaning log; mengomunikasikan hasil dalam format eksekutif; self-review dengan rubrik senior
Prerequisites	Bab 1-7 selesai beserta exercises
Estimasi waktu	Proyek 12-20 jam (1-2 minggu part-time)
Difficulty	● Advanced (integrasi) → ● Production Ready
Skills covered	Integrasi penuh Layer 1 + dokumentasi + komunikasi stakeholder
Business domain	E-commerce/retail (dataset publik nyata)

Mengapa capstone berbentuk begini

Ujian sesungguhnya dari Layer 1 bukan "bisakah kamu menulis window function", melainkan: diberi data mentah yang kotor dan pertanyaan bisnis yang kabur, bisakah kamu menghasilkan dataset yang orang lain berani pakai — dan menjelaskannya dalam bahasa yang eksekutif pahami?

Capstone ini meniru minggu-minggu pertama pekerjaan analyst sungguhan, termasuk bagian yang jarang dilatih: dokumentasi dan komunikasi.

Skenario

Kamu analyst baru di sebuah e-commerce. Head of CRM berkata: *"Kita mau mulai program retensi kuartal depan. Datanya berantakan dan aku nggak tahu mulai dari mana. Tolong siapkan fondasi datanya, dan kasih tahu aku apa yang kamu temukan."*

Permintaan ini sengaja kabur — persis seperti dunia nyata. Tugas pertamamu adalah Bab 1: framing.

Dataset yang disarankan

Gunakan salah satu dataset publik nyata (keduanya kotor secara autentik): **Olist Brazilian E-commerce** (Kaggle — multi-tabel: orders, items, customers, payments, reviews; sangat mirip skema TokoNusa) atau **Online Retail II** (UCI — satu tabel penuh retur dan anomali). Olist lebih disarankan: relasinya menguji skill JOIN dan grain.

Deliverables — enam artefak

#	Artefak	Menguji bab
D1	Analysis brief 1 halaman (framing + scope)	Bab 1
D2	Data audit report: grain, relasi, temuan 6 dimensi integrity	Bab 2, 6
D3	Script SQL + Python reproducible: raw → analytical dataset	Bab 3, 4, 6, 7
D4	Analytical dataset: customer-level (grain: 1 pelanggan) dengan RFM + feature	Bab 5, 6

D5	Data dictionary + cleaning log + 4 data test	Bab 5, 6
D6	Executive summary 1 halaman + lampiran teknis	Bab 1, 8

Spesifikasi D4 — analytical dataset minimum

Grain: **1 baris = 1 pelanggan**. Kolom minimum:

customer_id	first_order_date	recency_days
n_orders	last_order_date	frequency_score (1-5)
total_net_revenue	tenure_days	monetary_score (1-5)
avg_order_value	preferred_category	rfm_segment
n_categories	preferred_payment	is_single_purchase (flag)
return_rate	data_quality_flags	review_avg (jika Olist)

Setiap kolom harus punya baris di data dictionary: definisi bisnis satu kalimat, sumber tabel, aturan NULL, dan cara validasinya. Contoh format:

```

kolom: recency_days
definisi : hari sejak order completed terakhir hingga snapshot
           (snapshot_date dideklarasikan di header dataset)
sumber   : orders.order_ts (status = completed saja)
null      : tidak boleh NULL – pelanggan tanpa order completed
           tidak masuk dataset ini (lihat cleaning log #4)
validasi  : min ≥ 0; max ≤ tenure_days

```

Alur kerja yang disarankan

Minggu 1

D1 Framing (2 jam)
D2 Audit & profiling (3 jam)
D3 Extract & cleaning (4 jam)
└─ checkpoint: total revenue
 rekonsiliasi vs agregat kasar

Minggu 2

D4 Feature & RFM (3 jam)
D5 Dokumentasi & test (2 jam)
D6 Komunikasi (2 jam)
└─ self-review rubrik
 + revisi

D6 — Komunikasi: keterampilan yang menggandakan nilai semua yang lain

Mengapa

Head of CRM tidak akan membaca notebook-mu. Ia akan membaca satu halaman — dan dari halaman itu memutuskan apakah pekerjaanmu dipercaya. Analisis yang tidak terkomunikasikan secara efektif, secara bisnis, tidak pernah terjadi.

Struktur executive summary (pyramid principle)

- ANSWER FIRST — jawaban/temuan utama dalam 2-3 kalimat
"Fondasi data retensi siap. 3 temuan penting: (a) 62% pelanggan hanya membeli sekali; (b) segmen At-Risk bernilai 18% revenue dan bisa ditarget sekarang; (c) 9% record pelanggan bermasalah kualitas — sudah ditangani dan didokumentasikan."
- SUPPORTING — 3-5 bullet bukti, satu angka kunci per bullet

3. SO WHAT — rekomendasi tindakan + apa yang kamu butuhkan
4. APPENDIX — metodologi, definisi, keterbatasan (untuk yang bertanya)

✓ **BEST PRACTICE — Aturan komunikasi analyst**

- ✓ Kesimpulan di kalimat pertama, bukan di akhir
- ✓ Setiap angka penting punya pembanding (vs baseline, vs periode lalu)
- ✓ Keterbatasan dinyatakan sendiri sebelum orang lain menemukannya
- ✓ Rekomendasi dalam bahasa tindakan ("targetkan segmen X dengan Y"), bukan bahasa data ("terdapat korelasi antara...")
- ✓ Definisi metric di footnote — selalu.

✗ **COMMON MISTAKE — Melaporkan proses, bukan temuan**

Gejala: laporan dibuka dengan "Pertama saya membersihkan data, lalu saya melakukan JOIN..." dan eksekutif berhenti membaca di paragraf dua.

Mengapa terjadi: analyst bangga pada kerja kerasnya (wajar), tetapi pembaca membayar untuk jawabannya.

Perbaikan: proses pindah ke appendix. Halaman pertama hanya: jawaban, bukti, tindakan.

Cara senior menghindarinya: menulis kalimat pertama TERAKHIR — setelah tahu persis apa pesan utamanya.

Rubrik penilaian — review seperti enam reviewer

Nilai dirimu sendiri dengan enam kaca mata. Standar lulus: minimal "baik" di semua baris.

Reviewer	Pertanyaan kunci	Lulus jika
Senior Data Analyst	Apakah grain konsisten dan JOIN bebas fan-out?	Rekonsiliasi total lolos; tidak ada dobel hitung
Analytics Manager	Bisakah orang lain melanjutkan pekerjaan ini?	Script jalan ujung-ke-ujung; dictionary & log lengkap
BI Consultant	Apakah dataset menjawab kebutuhan program retensi?	RFM + flag actionable; segmen bisa langsung ditarget
Hiring Manager	Apakah ini membuktikan kompetensi Layer 1?	Semua skill semester terpakai secara alami
Technical Editor	Apakah keputusan cleaning terdokumentasi & defensible?	Setiap drop/impute/flag punya alasan tertulis
Eksekutif	Apakah 1 halaman itu membuat saya tahu harus apa?	Jawaban dulu; angka berpembanding; rekomendasi jelas

Kesalahan capstone yang paling sering

- **Scope membengkak.** Ini proyek fondasi data, bukan analisis retensi penuh (itu Semester 2-3). Tahan godaan membangun dashboard.
- **Revenue tidak rekonsiliasi.** Total di dataset final harus cocok dengan agregat kasar data mentah (setelah dikurangi yang sengaja dikeluarkan — dan jumlah itu tercatat).
- **Membuang data yang "aneh" tanpa dokumentasi.** Reviewer teknis selalu menanyakan: dari 100 ribu order mentah, kenapa dataset-mu berisi 91 ribu? Kamu harus bisa menjawab per ribu-nya.
- **Executive summary berisi jargon.** "Median recency segmen 4-5-x adalah 12 hari" → seharusnya "pelanggan terbaik kita berbelanja rata-rata tiap 2 minggu".

Ending Card — Bab 8 & Penutup Semester 1

What you learned (semester)

Framing (1) → literacy (2) → SQL fundamental & produksi (3-4) → arsitektur & pipeline (5) → preparation & feature (6) → tooling (7) → integrasi & komunikasi (8). Output semester tercapai: **mampu mengubah data mentah menjadi analytical dataset yang dapat dipercaya** — dan menjelaskannya.

Portfolio readiness

Capstone yang lulus rubrik adalah proyek portofolio utuh: kasus nyata, data kotor nyata, keputusan terdokumentasi, komunikasi eksekutif. Bersama artefak bab-bab sebelumnya, kamu kini punya repositori yang menceritakan kompetensi penuh Layer 1.

Interview readiness (semester)

● SIMULASI INTERVIEW LAYER 1 — uji dirimu

1. Jelaskan grain dan berikan contoh kesalahan yang terjadi jika mengabaikannya. (Bab 2)
2. Live SQL: top-3 per grup + growth MoM. (Bab 4)
3. "Dashboard anjlok 40% kemarin — 15 menit pertamamu?" (Bab 5)
4. "34% kolom penting kosong — apa yang kamu lakukan?" (Bab 6)
5. "Merge pandas menggandakan baris — kenapa dan bagaimana mencegahnya?" (Bab 7)
6. "Ceritakan proyek datamu" — jawab dengan struktur: masalah bisnis → keputusan kunci → dampak. (Bab 8)

Jika keenam jawaban keluar lancar dengan contoh konkret, kamu siap interview untuk posisi junior-mid data analyst di sisi data engineering-nya.

Menuju Semester 2

Dataset-mu kini bersih dan terpercaya. Pertanyaan berikutnya: *apa yang sebenarnya dikatakan data ini — dan seberapa yakin kita?* Volume 2 membangun mesin inferensi: descriptive statistics, probability, hypothesis testing, regression — plus visualization & BI dashboard yang mengubah statistik menjadi alat monitoring. Sampai jumpa di Layer 2.